

1000 SQL Practice Questions

JOINS • GROUP BY • Window Functions • CTEs • Date/String • DML • Schema • Data Engineering

242

Easy

465

Medium

293

Hard

10

Topics

1000

Total

Topics covered

1. JOINS (165 questions)
2. GROUP BY / HAVING (136 questions)
3. Window Functions (136 questions)
4. CTEs & Subqueries (107 questions)
5. Date & Time (107 questions)
6. String Functions (107 questions)
7. CASE & Logic (68 questions)
8. DML (INSERT/UPDATE/DELETE) (68 questions)
9. Schema & Indexes (67 questions)
10. Data Engineering (39 questions)

JOINS

Easy

Q1. Find all customers who have never placed an order.

```
SELECT c.* FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id WHERE o.order_id IS NULL;
```

Tip: LEFT JOIN + filter NULLs on the right side.

Easy

Q2. List every employee along with their department name.

```
SELECT e.name, d.dept_name FROM Employees e INNER JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: INNER JOIN returns only matched rows.

Medium

Q3. Find employees who earn more than their direct manager.

```
SELECT e.name, e.salary, m.name AS manager FROM Employees e JOIN Employees m ON e.manager_id = m.emp_id WHERE e.salary > m.salary;
```

Tip: Self-join on manager_id = emp_id.

Easy

Q4. Find all departments that currently have zero employees.

```
SELECT d.dept_name FROM Departments d LEFT JOIN Employees e ON d.dept_id = e.dept_id WHERE e.emp_id IS NULL;
```

Tip: LEFT JOIN + NULL filter.

Medium

Q5. Show each customer's name and their total order revenue.

```
SELECT c.first_name, SUM(o.total_amount) AS total_revenue FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id, c.first_name;
```

Tip: JOIN then GROUP BY + SUM.

Medium

Q6. Find all customers who have placed more than 3 orders.

```
SELECT c.first_name, COUNT(o.order_id) AS order_count FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING COUNT(o.order_id) > 3;
```

Tip: HAVING filters after GROUP BY.

Hard

Q7. Find the department-wise highest-paid employee including their name.

```
SELECT d.dept_name, e.name, e.salary FROM Employees e JOIN Departments d ON e.dept_id = d.dept_id WHERE e.salary = ( SELECT MAX(salary) FROM Employees e2 WHERE e2.dept_id = e.dept_id );
```

Tip: Correlated subquery for max salary per dept.

Easy

Q8. List all orders placed by customers from the USA.

```
SELECT o.* FROM Orders o JOIN Customers c ON o.cust_id = c.cust_id WHERE c.country = 'USA';
```

Tip: Filter after JOIN on country.

Medium

Q9. Find products that have never been ordered.

```
SELECT p.prod_name FROM Products p LEFT JOIN OrderItems oi ON p.prod_id = oi.prod_id WHERE oi.prod_id IS NULL;
```

Tip: LEFT JOIN + NULL check.

Hard

Q10. List all pairs of employees who work in the same department.

```
SELECT a.name AS emp1, b.name AS emp2 FROM Employees a JOIN Employees b ON a.dept_id = b.dept_id AND a.emp_id < b.emp_id;
```

Tip: Self-join with < to avoid duplicate pairs.

Medium Q11. Find customers who have purchased Electronics products.

```
SELECT DISTINCT c.first_name FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id JOIN OrderItems oi ON o.order_id = oi.order_id JOIN Products p ON oi.prod_id = p.prod_id WHERE p.category = 'Electronics';
```

Tip: Chain three JOINS across four tables.

Easy Q12. Find employees whose manager_id is NULL.

```
SELECT name FROM Employees WHERE manager_id IS NULL;
```

Tip: Simple WHERE NULL check.

Medium Q13. Find duplicate customer records by full name.

```
SELECT first_name, COUNT(*) AS cnt FROM Customers GROUP BY first_name HAVING COUNT(*) > 1;
```

Tip: GROUP BY + HAVING COUNT > 1.

Medium Q14. Find orders that reference a non-existent customer.

```
SELECT o.* FROM Orders o LEFT JOIN Customers c ON o.cust_id = c.cust_id WHERE c.cust_id IS NULL;
```

Tip: Orphan detection via LEFT JOIN.

Easy Q15. Show each department's average salary.

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary FROM Departments d JOIN Employees e ON d.dept_id = e.dept_id GROUP BY d.dept_name;
```

Tip: JOIN + GROUP BY + AVG.

Easy Q16. Find total sales grouped by customer country.

```
SELECT c.country, SUM(o.total_amount) AS total_sales FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.country;
```

Tip: JOIN + GROUP BY country.

Hard Q17. Find the single top customer by total revenue.

```
SELECT c.first_name, SUM(o.total_amount) AS revenue FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id ORDER BY revenue DESC LIMIT 1;
```

Tip: ORDER BY + LIMIT 1.

Medium Q18. Find customers whose last order was placed in 2023.

```
SELECT c.first_name FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(YEAR(o.order_date)) = 2023;
```

Tip: MAX(YEAR()) in HAVING.

Medium Q19. Find each employee's name and their manager's department name.

```
SELECT e.name, dm.dept_name AS manager_dept FROM Employees e JOIN Employees m ON e.manager_id = m.emp_id JOIN Departments dm ON m.dept_id = dm.dept_id;
```

Tip: Self-join + join to departments.

Hard Q20. Identify customers who have ordered from every product category.

```
SELECT c.first_name FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id JOIN OrderItems oi ON o.order_id = oi.order_id JOIN Products p ON oi.prod_id = p.prod_id GROUP BY c.cust_id HAVING COUNT(DISTINCT p.category) = (SELECT COUNT(DISTINCT category) FROM Products);
```

Tip: Compare distinct categories per customer to total.

GROUP BY / HAVING

Easy

Q21. Count the number of orders placed by each customer.

```
SELECT cust_id, COUNT(*) AS order_count FROM Orders GROUP BY cust_id;
```

Tip: GROUP BY + COUNT.

Medium

Q22. Find customers whose total spending exceeds \$5000.

```
SELECT cust_id, SUM(total_amount) AS total FROM Orders GROUP BY cust_id HAVING SUM(total_amount) > 5000;
```

Tip: HAVING on aggregated SUM.

Easy

Q23. Calculate average salary per department.

```
SELECT dept_id, AVG(salary) FROM Employees GROUP BY dept_id;
```

Tip: GROUP BY + AVG.

Medium

Q24. Find departments where average salary exceeds \$60,000.

```
SELECT dept_id, AVG(salary) FROM Employees GROUP BY dept_id HAVING AVG(salary) > 60000;
```

Tip: HAVING with AVG comparison.

Medium

Q25. Calculate total inventory value per product category.

```
SELECT category, SUM(price * stock_qty) AS inventory_value FROM Products GROUP BY category;
```

Tip: SUM of a computed column per group.

Medium

Q26. Find product categories with more than 10 products.

```
SELECT category, COUNT(*) FROM Products GROUP BY category HAVING COUNT(*) > 10;
```

Tip: COUNT + HAVING.

Easy

Q27. Find countries with more than 5 customers.

```
SELECT country, COUNT(*) FROM Customers GROUP BY country HAVING COUNT(*) > 5;
```

Tip: GROUP BY country + HAVING.

Medium

Q28. Calculate total sales for each calendar month.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total_amount) AS sales FROM Orders GROUP BY month;
```

Tip: DATE_FORMAT + GROUP BY.

Easy

Q29. Find year-wise total sales.

```
SELECT YEAR(order_date) AS yr, SUM(total_amount) FROM Orders GROUP BY yr;
```

Tip: YEAR() + GROUP BY.

Medium

Q30. Find the maximum order amount per customer.

```
SELECT cust_id, MAX(total_amount) FROM Orders GROUP BY cust_id;
```

Tip: MAX per group.

Easy

Q31. Find the minimum salary in each department.

```
SELECT dept_id, MIN(salary) FROM Employees GROUP BY dept_id;
```

Tip: MIN + GROUP BY.

Hard Q32. Find departments where total salary spend exceeds \$100,000.

```
SELECT dept_id, SUM(salary) FROM Employees GROUP BY dept_id HAVING SUM(salary) > 100000;
```

Tip: SUM + HAVING threshold.

Medium Q33. Find average product price per category where average > \$50.

```
SELECT category, AVG(price) FROM Products GROUP BY category HAVING AVG(price) > 50;
```

Tip: HAVING on AVG(price).

Easy Q34. Count distinct customers per country.

```
SELECT country, COUNT(DISTINCT cust_id) FROM Customers GROUP BY country;
```

Tip: COUNT DISTINCT inside GROUP BY.

Medium Q35. Find customers whose average order value exceeds \$500.

```
SELECT cust_id, AVG(total_amount) FROM Orders GROUP BY cust_id HAVING AVG(total_amount) > 500;
```

Tip: AVG per customer, HAVING filter.

Medium Q36. Find months where total sales exceeded \$10,000.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total_amount) AS sales FROM Orders GROUP BY month HAVING SUM(total_amount) > 10000;
```

Tip: Month group + HAVING SUM.

Medium Q37. Find categories where total stock quantity is below 50.

```
SELECT category, SUM(stock_qty) FROM Products GROUP BY category HAVING SUM(stock_qty) < 50;
```

Tip: SUM stock, HAVING < 50.

Easy Q38. Count the number of employees per manager.

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id;
```

Tip: COUNT + GROUP BY manager.

Easy Q39. Count orders placed in each month.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, COUNT(*) FROM Orders GROUP BY month;
```

Tip: DATE_FORMAT + GROUP BY.

Medium Q40. Find products that share the same name (duplicates).

```
SELECT prod_name, COUNT(*) FROM Products GROUP BY prod_name HAVING COUNT(*) > 1;
```

Tip: GROUP BY + HAVING COUNT > 1.

Window Functions

Medium Q41. Rank all employees by salary from highest to lowest.

```
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS salary_rank FROM Employees;
```

Tip: RANK() with ORDER BY in the window.

Medium Q42. Rank employees by salary within each department.

```
SELECT name, dept_id, salary, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM Employees;
```

Tip: *PARTITION BY* adds per-group ranking.

Hard **Q43. Find the top 2 earners per department.**

```
SELECT * FROM ( SELECT name, dept_id, salary, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY
salary DESC) AS rnk FROM Employees ) t WHERE rnk <= 2;
```

Tip: *Wrap window in subquery, then filter rnk <= 2.*

Medium **Q44. Find the second highest salary using DENSE_RANK.**

```
SELECT salary FROM ( SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM Employees )
t WHERE rnk = 2 LIMIT 1;
```

Tip: *DENSE_RANK* skips no ranks for ties.

Medium **Q45. Find the most recent order per customer.**

```
SELECT * FROM ( SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date DESC) AS rn
FROM Orders ) t WHERE rn = 1;
```

Tip: *ROW_NUMBER + PARTITION BY, DESC date.*

Medium **Q46. Find the very first order per customer.**

```
SELECT * FROM ( SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date ASC) AS rn
FROM Orders ) t WHERE rn = 1;
```

Tip: *Same pattern, ASC date order.*

Medium **Q47. Calculate a running total of sales ordered by date.**

```
SELECT order_date, total_amount, SUM(total_amount) OVER (ORDER BY order_date ROWS UNBOUNDED
PRECEDING) AS running_total FROM Orders;
```

Tip: *SUM with ROWS UNBOUNDED PRECEDING.*

Medium **Q48. Calculate running cumulative stock by product ID.**

```
SELECT prod_id, stock_qty, SUM(stock_qty) OVER (ORDER BY prod_id ROWS UNBOUNDED PRECEDING) AS
cum_stock FROM Products;
```

Tip: *Running sum over prod_id ordering.*

Medium **Q49. Use LAG to compare each order with the previous order.**

```
SELECT order_id, total_amount, LAG(total_amount) OVER (PARTITION BY cust_id ORDER BY order_date) AS
prev, total_amount - LAG(total_amount) OVER (PARTITION BY cust_id ORDER BY order_date) AS diff FROM
Orders;
```

Tip: *LAG(col,1) looks back one row.*

Medium **Q50. Use LEAD to compare each salary with the next one.**

```
SELECT name, salary, LEAD(salary) OVER (ORDER BY salary) AS next_salary FROM Employees;
```

Tip: *LEAD(col,1) looks forward one row.*

Hard **Q51. Calculate month-over-month sales growth percentage.**

```
WITH monthly AS ( SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total_amount) AS sales FROM
Orders GROUP BY month ) SELECT month, sales, ROUND(100.0 * (sales - LAG(sales) OVER (ORDER BY
month)) / LAG(sales) OVER (ORDER BY month), 2) AS pct_growth FROM monthly;
```

Tip: *CTE for monthly totals, then LAG for MoM.*

Medium Q52. Find each customer's first and last order amounts.

```
SELECT cust_id, FIRST_VALUE(total_amount) OVER (PARTITION BY cust_id ORDER BY order_date) AS first_order, LAST_VALUE(total_amount) OVER (PARTITION BY cust_id ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_order FROM Orders;
```

Tip: LAST_VALUE needs explicit frame clause.

Medium Q53. Identify duplicate rows using ROW_NUMBER.

```
SELECT * FROM ( SELECT *, ROW_NUMBER() OVER (PARTITION BY first_name, country ORDER BY cust_id) AS rn FROM Customers ) t WHERE rn > 1;
```

Tip: Rows with rn > 1 are duplicates.

Hard Q54. Delete duplicate rows keeping the lowest ID.

```
DELETE FROM Customers WHERE cust_id NOT IN ( SELECT MIN(cust_id) FROM Customers GROUP BY first_name, country );
```

Tip: Keep only MIN(id) per duplicate group.

Hard Q55. Find the top 3 most expensive products in each category.

```
SELECT * FROM ( SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY price DESC) AS rnk FROM Products ) t WHERE rnk <= 3;
```

Tip: DENSE_RANK per category, filter <= 3.

Hard Q56. Calculate cumulative salary spend per department.

```
SELECT dept_id, name, salary, SUM(salary) OVER (PARTITION BY dept_id ORDER BY emp_id ROWS UNBOUNDED PRECEDING) AS cum_salary FROM Employees;
```

Tip: Running SUM partitioned by dept.

Hard Q57. Compute a 3-order moving average of sales per customer.

```
SELECT order_id, cust_id, total_amount, AVG(total_amount) OVER (PARTITION BY cust_id ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg FROM Orders;
```

Tip: ROWS BETWEEN 2 PRECEDING AND CURRENT ROW.

Medium Q58. Divide employees into 4 salary quartiles using NTILE.

```
SELECT name, salary, NTILE(4) OVER (ORDER BY salary DESC) AS quartile FROM Employees;
```

Tip: NTILE(4) splits into four equal buckets.

Hard Q59. Calculate each order's percentage of total sales.

```
SELECT order_id, total_amount, ROUND(100.0 * total_amount / SUM(total_amount) OVER (), 2) AS pct_of_total FROM Orders;
```

Tip: SUM() OVER () without PARTITION = grand total.

Hard Q60. Find employees earning above their department's average.

```
SELECT * FROM ( SELECT name, dept_id, salary, AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg FROM Employees ) t WHERE salary > dept_avg;
```

Tip: Can't use window fn in WHERE; wrap in subquery.

CTEs & Subqueries

Medium Q61. Find employees who earn more than the company average salary.

```
SELECT name, salary FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees);
```

Tip: Scalar subquery in WHERE.

Hard **Q62. Find employees earning more than their department average.**

```
WITH dept_avg AS ( SELECT dept_id, AVG(salary) AS avg_sal FROM Employees GROUP BY dept_id ) SELECT  
e.name, e.salary, da.avg_sal FROM Employees e JOIN dept_avg da ON e.dept_id = da.dept_id WHERE  
e.salary > da.avg_sal;
```

Tip: CTE precomputes dept averages, then JOIN.

Medium **Q63. Find the second highest salary without using RANK.**

```
SELECT MAX(salary) AS second_highest FROM Employees WHERE salary < (SELECT MAX(salary) FROM  
Employees);
```

Tip: Nested MAX: exclude top value then MAX again.

Medium **Q64. Find customers whose total orders exceed average customer spend.**

```
SELECT cust_id, SUM(total_amount) AS total FROM Orders GROUP BY cust_id HAVING SUM(total_amount) > (  
SELECT AVG(cust_total) FROM ( SELECT SUM(total_amount) AS cust_total FROM Orders GROUP BY cust_id )  
sub );
```

Tip: Nested subquery computes avg of per-customer totals.

Medium **Q65. Find products priced above their category's average price.**

```
SELECT prod_name, category, price FROM Products p WHERE price > ( SELECT AVG(price) FROM Products p2  
WHERE p2.category = p.category );
```

Tip: Correlated subquery compares within category.

Medium **Q66. Find departments with no employees using NOT EXISTS.**

```
SELECT dept_name FROM Departments d WHERE NOT EXISTS ( SELECT 1 FROM Employees e WHERE e.dept_id =  
d.dept_id );
```

Tip: NOT EXISTS returns true when subquery finds nothing.

Medium **Q67. Find customers who placed at least one order using EXISTS.**

```
SELECT first_name FROM Customers c WHERE EXISTS ( SELECT 1 FROM Orders o WHERE o.cust_id = c.cust_id  
);
```

Tip: EXISTS stops at first matching row.

Medium **Q68. Find customers who never ordered using NOT EXISTS.**

```
SELECT first_name FROM Customers c WHERE NOT EXISTS ( SELECT 1 FROM Orders o WHERE o.cust_id =  
c.cust_id );
```

Tip: Complement of EXISTS pattern.

Medium **Q69. Find products that have never been sold using a subquery.**

```
SELECT prod_name FROM Products WHERE prod_id NOT IN ( SELECT DISTINCT prod_id FROM OrderItems );
```

Tip: NOT IN; prefer NOT EXISTS if NULLs possible.

Medium **Q70. Find each customer's latest order using a CTE.**

```
WITH latest AS ( SELECT cust_id, MAX(order_date) AS last_order FROM Orders GROUP BY cust_id ) SELECT  
c.first_name, l.last_order FROM Customers c JOIN latest l ON c.cust_id = l.cust_id;
```

Tip: CTE isolates aggregation for readability.

Medium **Q71. Find duplicate products using a CTE.**


```
WITH dupes AS ( SELECT prod_name, COUNT(*) AS cnt FROM Products GROUP BY prod_name ) SELECT * FROM dupes WHERE cnt > 1;
```

Tip: CTE + filter on count.

Medium Q72. Calculate monthly revenue using a CTE.

```
WITH monthly AS ( SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total_amount) AS revenue FROM Orders GROUP BY month ) SELECT * FROM monthly ORDER BY month;
```

Tip: Simple aggregation CTE.

Hard Q73. Find months where revenue is above average monthly revenue.

```
WITH monthly AS ( SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total_amount) AS revenue FROM Orders GROUP BY month ) SELECT * FROM monthly WHERE revenue > (SELECT AVG(revenue) FROM monthly);
```

Tip: Reference the same CTE twice.

Medium Q74. Find the top revenue customer using a CTE.

```
WITH cust_rev AS ( SELECT cust_id, SUM(total_amount) AS total FROM Orders GROUP BY cust_id ) SELECT c.first_name, cr.total FROM Customers c JOIN cust_rev cr ON c.cust_id = cr.cust_id ORDER BY cr.total DESC LIMIT 1;
```

Tip: CTE + JOIN + ORDER + LIMIT 1.

Medium Q75. Find categories where average price is above \$50 using a CTE.

```
WITH cat_avg AS ( SELECT category, AVG(price) AS avg_price FROM Products GROUP BY category ) SELECT * FROM cat_avg WHERE avg_price > 50;
```

Tip: CTE filters on aggregated value.

Hard Q76. Find employees whose salary exceeds their manager's salary.

```
SELECT e.name, e.salary, m.salary AS mgr_salary FROM Employees e JOIN Employees m ON e.manager_id = m.emp_id WHERE e.salary > m.salary;
```

Tip: Self-join pattern.

Hard Q77. Find customers who bought from every product category.

```
SELECT cust_id FROM ( SELECT DISTINCT o.cust_id, p.category FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id JOIN Products p ON oi.prod_id = p.prod_id ) sub GROUP BY cust_id HAVING COUNT(DISTINCT category) = (SELECT COUNT(DISTINCT category) FROM Products);
```

Tip: Count distinct categories vs total categories.

Medium Q78. Find products priced above the overall product average.

```
SELECT prod_name, price FROM Products WHERE price > (SELECT AVG(price) FROM Products);
```

Tip: Simple scalar subquery.

Hard Q79. Find departments where the max salary is above company average.

```
SELECT dept_id, MAX(salary) FROM Employees GROUP BY dept_id HAVING MAX(salary) > (SELECT AVG(salary) FROM Employees);
```

Tip: HAVING with scalar subquery.

Medium Q80. Find orders whose amount is above the average order value.

```
SELECT * FROM Orders WHERE total_amount > (SELECT AVG(total_amount) FROM Orders);
```

Tip: Scalar subquery for average.

Date & Time

Easy Q81. Find all orders placed in the last 7 days.

```
SELECT * FROM Orders WHERE order_date >= CURDATE() - INTERVAL 7 DAY;
```

Tip: `CURDATE() - INTERVAL` is standard.

Easy Q82. Find all orders placed in the current month.

```
SELECT * FROM Orders WHERE MONTH(order_date) = MONTH(CURDATE()) AND YEAR(order_date) = YEAR(CURDATE());
```

Tip: Match both month and year.

Easy Q83. Find all orders placed during the year 2024.

```
SELECT * FROM Orders WHERE YEAR(order_date) = 2024;
```

Tip: `YEAR()` function filter.

Easy Q84. Extract year, month, and day from every order date.

```
SELECT order_id, YEAR(order_date) AS yr, MONTH(order_date) AS mo, DAY(order_date) AS dy FROM Orders;
```

Tip: `YEAR()`, `MONTH()`, `DAY()` functions.

Medium Q85. Find employees hired within the last 12 months.

```
SELECT * FROM Employees WHERE hire_date >= CURDATE() - INTERVAL 1 YEAR;
```

Tip: `INTERVAL 1 YEAR` subtraction.

Medium Q86. Find orders placed between Jan 1 and Mar 31, 2024.

```
SELECT * FROM Orders WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';
```

Tip: `BETWEEN` is inclusive on both ends.

Hard Q87. Write an incremental load query using an `updated_date` column.

```
SELECT * FROM SourceTable WHERE updated_date > ( SELECT MAX(updated_date) FROM TargetTable );
```

Tip: Fetch only rows newer than the target watermark.

Medium Q88. Find the month in 2023 with the highest total sales.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total_amount) AS sales FROM Orders WHERE YEAR(order_date) = 2023 GROUP BY month ORDER BY sales DESC LIMIT 1;
```

Tip: Filter year, `GROUP BY` month, `ORDER + LIMIT`.

Medium Q89. Find orders placed on a weekend.

```
SELECT * FROM Orders WHERE DAYOFWEEK(order_date) IN (1, 7);
```

Tip: 1=Sunday, 7=Saturday in MySQL.

Medium Q90. Calculate the number of days between order date and today.

```
SELECT order_id, DATEDIFF(CURDATE(), order_date) AS days_ago FROM Orders;
```

Tip: `DATEDIFF(end, start)` returns integer days.

Hard Q91. Find customers who placed orders in both 2022 and 2023.

```
SELECT cust_id FROM Orders WHERE YEAR(order_date) IN (2022, 2023) GROUP BY cust_id HAVING COUNT(DISTINCT YEAR(order_date)) = 2;
```

Tip: Count distinct years per customer.

Medium Q92. Find orders placed on the last day of each month.

```
SELECT * FROM Orders WHERE DAY(order_date) = DAY(LAST_DAY(order_date));
```

Tip: LAST_DAY() returns last day of the month.

Medium Q93. Find total sales per quarter for 2023.

```
SELECT QUARTER(order_date) AS qtr, SUM(total_amount) FROM Orders WHERE YEAR(order_date) = 2023 GROUP BY qtr;
```

Tip: QUARTER() extracts Q1-Q4.

Hard Q94. Calculate age of each order in weeks.

```
SELECT order_id, FLOOR(DATEDIFF(CURDATE(), order_date) / 7) AS age_weeks FROM Orders;
```

Tip: Divide DATEDIFF by 7 and floor.

Medium Q95. Find the average days between a customer's consecutive orders.

```
WITH ordered AS ( SELECT cust_id, order_date, LAG(order_date) OVER (PARTITION BY cust_id ORDER BY order_date) AS prev_date FROM Orders ) SELECT cust_id, AVG(DATEDIFF(order_date, prev_date)) AS avg_days FROM ordered WHERE prev_date IS NOT NULL GROUP BY cust_id;
```

Tip: LAG for previous date, then DATEDIFF, then AVG.

Medium Q96. Find customers who placed an order on their account creation date.

```
SELECT c.first_name FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE DATE(o.order_date) = DATE(c.created_at);
```

Tip: DATE() strips the time part.

Easy Q97. Find orders where delivery date is before the order date (data error).

```
SELECT * FROM Orders WHERE delivery_date < order_date;
```

Tip: Simple column comparison.

Medium Q98. Find the first order date for each customer.

```
SELECT cust_id, MIN(order_date) AS first_order FROM Orders GROUP BY cust_id;
```

Tip: MIN on date per group.

Hard Q99. Find customers who have not ordered in the last 90 days.

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id AND o.order_date >= CURDATE() - INTERVAL 90 DAY WHERE o.order_id IS NULL;
```

Tip: LEFT JOIN with date condition, NULL = no recent order.

Medium Q100. Compare total revenue for last 30 days vs prior 30 days.

```
SELECT SUM(CASE WHEN order_date >= CURDATE()-INTERVAL 30 DAY THEN total_amount ELSE 0 END) AS last_30, SUM(CASE WHEN order_date BETWEEN CURDATE()-INTERVAL 60 DAY AND CURDATE()-INTERVAL 31 DAY THEN total_amount ELSE 0 END) AS prior_30 FROM Orders;
```

Tip: Conditional SUM with CASE for two date windows.

String Functions

Easy

Q101. Find all customers whose name starts with 'A'.

```
SELECT * FROM Customers WHERE first_name LIKE 'A%';
```

Tip: LIKE with trailing wildcard.

Easy

Q102. Find all customers whose name ends with 'n'.

```
SELECT * FROM Customers WHERE first_name LIKE '%n';
```

Tip: LIKE with leading wildcard.

Easy

Q103. Find customers whose name contains 'ar'.

```
SELECT * FROM Customers WHERE first_name LIKE '%ar%';
```

Tip: LIKE with wildcards on both sides.

Easy

Q104. Convert all customer names to uppercase.

```
SELECT UPPER(first_name) AS name_upper FROM Customers;
```

Tip: UPPER() function.

Easy

Q105. Replace NULL country values with 'Unknown'.

```
SELECT first_name, COALESCE(country, 'Unknown') AS country FROM Customers;
```

Tip: COALESCE returns first non-NULL argument.

Easy

Q106. Find products where the product name is NULL.

```
SELECT * FROM Products WHERE prod_name IS NULL;
```

Tip: IS NULL — cannot use = NULL.

Easy

Q107. Display product names, replacing NULL with 'Unnamed Product'.

```
SELECT COALESCE(prod_name, 'Unnamed Product') AS prod_name FROM Products;
```

Tip: COALESCE replacement.

Medium

Q108. Trim leading and trailing spaces from customer names.

```
SELECT TRIM(first_name) AS clean_name FROM Customers;
```

Tip: TRIM() removes whitespace by default.

Medium

Q109. Find the length of each product name.

```
SELECT prod_name, LENGTH(prod_name) AS name_len FROM Products;
```

Tip: LENGTH() returns character count.

Medium

Q110. Concatenate first and last name into a full name column.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM Customers;
```

Tip: CONCAT with a space separator.

Medium

Q111. Find customers whose email domain is gmail.com.

```
SELECT * FROM Customers WHERE email LIKE '%@gmail.com';
```

Tip: LIKE pattern for domain.

Medium

Q112. Extract the domain part from an email address.

```
SELECT email, SUBSTRING_INDEX(email, '@', -1) AS domain FROM Customers;
```

Tip: SUBSTRING_INDEX with delimiter '@', -1 gets part after.

Medium Q113. Reverse all product names.

```
SELECT prod_name, REVERSE(prod_name) AS reversed FROM Products;
```

Tip: `REVERSE()` function.

Medium Q114. Find products whose name contains a digit.

```
SELECT * FROM Products WHERE prod_name REGEXP '[0-9]';
```

Tip: `REGEXP` with character class.

Medium Q115. Replace underscores with spaces in product names.

```
SELECT REPLACE(prod_name, '_', ' ') AS clean_name FROM Products;
```

Tip: `REPLACE(col, find, replacement)`.

Easy Q116. Convert country names to lowercase.

```
SELECT LOWER(country) AS country_lower FROM Customers;
```

Tip: `LOWER()` function.

Medium Q117. Find customers where phone number is not 10 digits.

```
SELECT * FROM Customers WHERE phone NOT REGEXP '^([0-9]{10})$';
```

Tip: Anchored `REGEXP` pattern.

Medium Q118. Pad order IDs with leading zeros to 8 characters.

```
SELECT LPAD(order_id, 8, '0') AS padded_id FROM Orders;
```

Tip: `LPAD(str, len, pad_char)`.

Hard Q119. Extract a value from a JSON string column.

```
SELECT JSON_UNQUOTE( JSON_EXTRACT(metadata_col, '$.key') ) AS extracted_val FROM SomeTable;
```

Tip: `JSON_EXTRACT` uses `JSONPath` syntax (MySQL 5.7+).

Medium Q120. Find employees whose name has exactly 5 characters.

```
SELECT * FROM Employees WHERE CHAR_LENGTH(name) = 5;
```

Tip: `CHAR_LENGTH` for multi-byte safe length.

CASE & Logic

Easy Q121. Label employees as 'High' if salary > 10000, else 'Low'.

```
SELECT name, salary, CASE WHEN salary > 10000 THEN 'High' ELSE 'Low' END AS salary_class FROM Employees;
```

Tip: Simple two-branch `CASE`.

Medium Q122. Create a three-tier salary label: High, Medium, Low.

```
SELECT name, salary, CASE WHEN salary > 10000 THEN 'High' WHEN salary >= 5000 THEN 'Medium' ELSE 'Low' END AS salary_tier FROM Employees;
```

Tip: Multiple `WHEN` branches evaluated top-down.

Easy Q123. Label orders as 'High Value' if amount > 1000, else 'Normal'.

```
SELECT order_id, total_amount, CASE WHEN total_amount > 1000 THEN 'High Value' ELSE 'Normal' END AS category FROM Orders;
```

Tip: CASE in SELECT list.

Medium Q124. Count high-value vs normal orders in one query.

```
SELECT SUM(CASE WHEN total_amount > 1000 THEN 1 ELSE 0 END) AS high_value, SUM(CASE WHEN total_amount <= 1000 THEN 1 ELSE 0 END) AS normal FROM Orders;
```

Tip: Conditional SUM — a common pivot trick.

Medium Q125. Use NULLIF to avoid division by zero.

```
SELECT cust_id, total_revenue / NULLIF(order_count, 0) AS avg_order_value FROM cust_summary;
```

Tip: NULLIF(expr,0) returns NULL when denominator is 0.

Medium Q126. Classify products: Expensive, Mid-range, or Budget.

```
SELECT prod_name, price, CASE WHEN price > 100 THEN 'Expensive' WHEN price > 50 THEN 'Mid-range' ELSE 'Budget' END AS price_class FROM Products;
```

Tip: Three-tier CASE.

Medium Q127. Create an activity flag: Active if ordered in last 90 days, else Inactive.

```
SELECT c.first_name, CASE WHEN MAX(o.order_date) >= CURDATE() - INTERVAL 90 DAY THEN 'Active' ELSE 'Inactive' END AS status FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CASE on an aggregated date.

Hard Q128. Pivot monthly sales into columns (Jan, Feb, Mar) using CASE.

```
SELECT YEAR(order_date) AS yr, SUM(CASE WHEN MONTH(order_date)=1 THEN total_amount ELSE 0 END) AS Jan, SUM(CASE WHEN MONTH(order_date)=2 THEN total_amount ELSE 0 END) AS Feb, SUM(CASE WHEN MONTH(order_date)=3 THEN total_amount ELSE 0 END) AS Mar FROM Orders GROUP BY yr;
```

Tip: Conditional SUM per month = manual pivot.

Medium Q129. Return the newer of two date columns using GREATEST.

```
SELECT order_id, GREATEST(created_at, updated_at) AS latest_date FROM Orders;
```

Tip: GREATEST() returns the largest value.

Medium Q130. Return 0 instead of NULL for missing salary values.

```
SELECT name, COALESCE(salary, 0) AS salary FROM Employees;
```

Tip: COALESCE replaces NULL with 0.

DML (INSERT/UPDATE/DELETE)

Easy Q131. Insert a new customer record.

```
INSERT INTO Customers (first_name, country) VALUES ('Alice', 'USA');
```

Tip: Basic INSERT with explicit column list.

Easy Q132. Insert multiple customers in one statement.

```
INSERT INTO Customers (first_name, country) VALUES ('Bob', 'UK'), ('Carla', 'France'), ('Dan', 'Germany');
```

Tip: Multi-row INSERT with comma-separated tuples.

Easy

Q133. Update the country for a specific customer.

```
UPDATE Customers SET country = 'Canada' WHERE cust_id = 42;
```

Tip: Always include *WHERE* to avoid full-table update.

Medium

Q134. Increase all Electronics prices by 10%.

```
UPDATE Products SET price = price * 1.10 WHERE category = 'Electronics';
```

Tip: Arithmetic in *SET* clause.

Medium

Q135. Delete all products with zero stock older than one year.

```
DELETE FROM Products WHERE stock_qty = 0 AND created_at < CURDATE() - INTERVAL 1 YEAR;
```

Tip: Compound *WHERE* condition.

Easy

Q136. Delete a specific order by ID.

```
DELETE FROM Orders WHERE order_id = 1001;
```

Tip: Single-row *DELETE*.

Hard

Q137. Upsert a customer using INSERT ... ON DUPLICATE KEY UPDATE.

```
INSERT INTO Customers (cust_id, first_name, country) VALUES (5, 'Eve', 'NZ') ON DUPLICATE KEY UPDATE first_name = VALUES(first_name), country = VALUES(country);
```

Tip: MySQL upsert pattern.

Medium

Q138. Copy records from staging to production avoiding duplicates.

```
INSERT INTO Customers (first_name, country) SELECT first_name, country FROM Customers_Staging s WHERE NOT EXISTS ( SELECT 1 FROM Customers c WHERE c.first_name = s.first_name );
```

Tip: *INSERT...SELECT* with *NOT EXISTS* deduplication.

Medium

Q139. Update salaries by joining to a raises table.

```
UPDATE Employees e JOIN Raises r ON e.emp_id = r.emp_id SET e.salary = e.salary + r.raise_amount;
```

Tip: Multi-table *UPDATE* using *JOIN*.

Hard

Q140. Soft-delete orders by setting a deleted_at timestamp.

```
UPDATE Orders SET deleted_at = NOW() WHERE order_id = 2001;
```

Tip: Soft delete — preserve data, mark with timestamp.

Schema & Indexes

Easy

Q141. Create a simple Customers table.

```
CREATE TABLE Customers ( cust_id INT PRIMARY KEY AUTO_INCREMENT, first_name VARCHAR(100) NOT NULL, country VARCHAR(50), created_at DATETIME DEFAULT NOW() );
```

Tip: Basic *CREATE TABLE* with common column types.

Easy

Q142. Add a new email column to the Customers table.

```
ALTER TABLE Customers ADD COLUMN email VARCHAR(150);
```

Tip: *ALTER TABLE ADD COLUMN*.

Easy

Q143. Create an index on the email column.

```
CREATE INDEX idx_customer_email ON Customers(email);
```

Tip: Non-unique index on single column.

Medium Q144. Create a composite index on country and created_at.

```
CREATE INDEX idx_country_created ON Customers(country, created_at);
```

Tip: Composite index — column order matters.

Medium Q145. Add a foreign key from Orders to Customers.

```
ALTER TABLE Orders ADD CONSTRAINT fk_orders_customer FOREIGN KEY (cust_id) REFERENCES Customers(cust_id);
```

Tip: Referential integrity with FK constraint.

Easy Q146. Drop a column that is no longer needed.

```
ALTER TABLE Customers DROP COLUMN middle_name;
```

Tip: ALTER TABLE DROP COLUMN.

Medium Q147. Add a unique constraint on email.

```
ALTER TABLE Customers ADD CONSTRAINT uq_customer_email UNIQUE(email);
```

Tip: UNIQUE constraint prevents duplicates.

Hard Q148. Create a partial index on active orders (PostgreSQL).

```
CREATE INDEX idx_active_orders ON Orders(cust_id) WHERE status = 'active';
```

Tip: Partial/filtered index — not all databases support this.

Medium Q149. Check which indexes exist on the Orders table.

```
SHOW INDEX FROM Orders;
```

Tip: MySQL-specific; use pg_indexes in PostgreSQL.

Easy Q150. Rename a column from 'name' to 'full_name'.

```
ALTER TABLE Employees RENAME COLUMN name TO full_name;
```

Tip: MySQL 8+ supports RENAME COLUMN directly.

Data Engineering

Medium Q151. Find records present in source but missing from target.

```
SELECT s.* FROM Source s LEFT JOIN Target t ON s.id = t.id WHERE t.id IS NULL;
```

Tip: LEFT JOIN + NULL filter to find missing rows.

Medium Q152. Compare row counts between source and target by date.

```
SELECT 'source' AS tbl, order_date, COUNT(*) AS cnt FROM Source GROUP BY order_date UNION ALL SELECT 'target', order_date, COUNT(*) FROM Target GROUP BY order_date ORDER BY order_date, tbl;
```

Tip: UNION ALL to stack both counts.

Hard Q153. Find rejected records where mandatory columns are NULL.

```
SELECT * FROM Orders WHERE order_id IS NULL OR cust_id IS NULL OR order_date IS NULL OR total_amount IS NULL;
```

Tip: OR chain of IS NULL checks.

Hard

Q154. Detect schema drift — columns in source not in target.

```
SELECT column_name FROM information_schema.columns WHERE table_name = 'Source' AND column_name NOT IN ( SELECT column_name FROM information_schema.columns WHERE table_name = 'Target' );
```

Tip: Use `information_schema` for metadata comparison.

Hard

Q155. Identify rows that changed between source and target (CDC).

```
SELECT s.* FROM Source s JOIN Target t ON s.id = t.id WHERE s.updated_at > t.updated_at OR s.checksum_col <> t.checksum_col;
```

Tip: Compare `updated_at` or `checksum` for changes.

Medium

Q156. Full outer join to find mismatches in both directions.

```
SELECT s.id AS src_id, t.id AS tgt_id FROM Source s FULL OUTER JOIN Target t ON s.id = t.id WHERE s.id IS NULL OR t.id IS NULL;
```

Tip: `FULL OUTER JOIN` reveals rows missing on either side.

Hard

Q157. Calculate data freshness in seconds.

```
SELECT TIMESTAMPDIFF(SECOND, MAX(updated_at), NOW()) AS seconds_stale FROM Target;
```

Tip: `TIMESTAMPDIFF` for freshness monitoring.

Medium

Q158. Deduplicate a staging table keeping the latest per business key.

```
DELETE FROM Staging WHERE id NOT IN ( SELECT id FROM ( SELECT id, ROW_NUMBER() OVER (PARTITION BY business_key ORDER BY updated_at DESC) AS rn FROM Staging ) t WHERE rn = 1 );
```

Tip: `ROW_NUMBER` to identify duplicates, delete non-first rows.

Hard

Q159. Detect rows with values more than 3 standard deviations from mean.

```
SELECT * FROM ( SELECT *, (total_amount - AVG(total_amount) OVER()) / STDDEV(total_amount) OVER() AS z_score FROM Orders ) t WHERE ABS(z_score) > 3;
```

Tip: Z-score using window `AVG` and `STDDEV`.

Hard

Q160. Validate referential integrity between Orders and Customers.

```
SELECT o.order_id FROM Orders o LEFT JOIN Customers c ON o.cust_id = c.cust_id WHERE c.cust_id IS NULL;
```

Tip: Orphan detection — orders with no matching customer.

JOINS

Easy

Q161. List all employees including those with no department, showing NULL as 'No Dept'.

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: `LEFT JOIN` + `COALESCE` for NULL dept.

Medium

Q162. Find employees who share the same salary as at least one other employee.

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with `<` to avoid duplicates.

Hard

Q163. Find customers who ordered but live in countries with fewer than 3 total customers.

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q164. Find the total number of products per category.

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q165. Find categories with total revenue above \$20,000.

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id =
oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount)
> 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q166. Find the department with the greatest salary variance.

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY
sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q167. Number each order within a customer partition by date.

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q168. Find the running average salary as employees are scanned by hire date.

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS
running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard

Q169. Calculate the salary percentile rank for each employee.

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q170. Using a CTE, list all customers who live in France.

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q171. Write a recursive CTE to show an employee org chart.

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q172. Find orders placed today.

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q173. Find the busiest day of the week for orders.

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q174. Find customers who have not placed an order in over 6 months.

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q175. Find customers whose names contain exactly 4 characters.

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q176. Find product names that are palindromes.

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q177. Return 'Yes' if an order is above \$500, else 'No'.

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q178. Update the stock quantity of a product after a sale.

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q179. Create a view that shows each customer's total order value.

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q180. Find columns with more than 5% NULL values (generic pattern).

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q181. Find all managers who have more than 5 direct reports.

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q182. Find customers who ordered the same product more than once.

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q183. Find the product with the most total units ordered.

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q184. Rank each order within a customer by amount, largest first.

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q185. Find products in the top 25% price within their category.

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: `NTILE(4)` then filter `quartile 1`.

String Functions

Medium Q186. Find employees whose names contain two consecutive vowels.

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: `REGEXP` with repetition quantifier.

CASE & Logic

Medium Q187. Bucket orders into quartiles by amount using `NTILE`.

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: `NTILE(4)` on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q188. Archive old orders to an archive table and delete from main.

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: `INSERT...SELECT` then `DELETE`.

Schema & Indexes

Hard Q189. Write a stored procedure to raise salary by a given percentage.

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with `IN` parameter.

JOINS

Easy Q190. List all employees including those with no department, showing `NULL` as 'No Dept'. (include only active records)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: `LEFT JOIN` + `COALESCE` for `NULL` dept.

Medium Q191. Find employees who share the same salary as at least one other employee. (include only active records)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q192. Find customers who ordered but live in countries with fewer than 3 total customers. (include only active records)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q193. Find the total number of products per category. (include only active records)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q194. Find categories with total revenue above \$20,000. (include only active records)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id =
oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount)
> 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q195. Find the department with the greatest salary variance. (include only active records)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY
sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q196. Number each order within a customer partition by date. (include only active records)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q197. Find the running average salary as employees are scanned by hire date. (include only active records)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS
running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard

Q198. Calculate the salary percentile rank for each employee. (include only active records)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q199. Using a CTE, list all customers who live in France. (include only active records)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q200. Write a recursive CTE to show an employee org chart. (include only active records)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q201. Find orders placed today. (include only active records)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q202. Find the busiest day of the week for orders. (include only active records)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard Q203. Find customers who have not placed an order in over 6 months. (include only active records)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy Q204. Find customers whose names contain exactly 4 characters. (include only active records)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium Q205. Find product names that are palindromes. (include only active records)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy Q206. Return 'Yes' if an order is above \$500, else 'No'. (include only active records)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium Q207. Update the stock quantity of a product after a sale. (include only active records)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q208. Create a view that shows each customer's total order value. (include only active records)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q209. Find columns with more than 5% NULL values (generic pattern). (include only active records)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable  
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q210. Find all managers who have more than 5 direct reports. (include only active records)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY  
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q211. Find customers who ordered the same product more than once. (include only active records)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =  
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q212. Find the product with the most total units ordered. (include only active records)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units  
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium

Q213. Rank each order within a customer by amount, largest first. (include only active records)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard

Q214. Find products in the top 25% price within their category. (include only active records)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium

Q215. Find employees whose names contain two consecutive vowels. (include only active records)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium

Q216. Bucket orders into quartiles by amount using NTILE. (include only active records)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard

Q217. Archive old orders to an archive table and delete from main. (include only active records)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard

Q218. Write a stored procedure to raise salary by a given percentage. (include only active records)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q219. List all employees including those with no department, showing NULL as 'No Dept'. (order results alphabetically)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q220. Find employees who share the same salary as at least one other employee. (order results alphabetically)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q221. Find customers who ordered but live in countries with fewer than 3 total customers. (order results alphabetically)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q222. Find the total number of products per category. (order results alphabetically)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q223. Find categories with total revenue above \$20,000. (order results alphabetically)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q224. Find the department with the greatest salary variance. (order results alphabetically)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q225. Number each order within a customer partition by date. (order results alphabetically)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q226. Find the running average salary as employees are scanned by hire date. (order results alphabetically)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q227. Calculate the salary percentile rank for each employee. (order results alphabetically)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q228. Using a CTE, list all customers who live in France. (order results alphabetically)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q229. Write a recursive CTE to show an employee org chart. (order results alphabetically)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q230. Find orders placed today. (order results alphabetically)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q231. Find the busiest day of the week for orders. (order results alphabetically)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard Q232. Find customers who have not placed an order in over 6 months. (order results alphabetically)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy Q233. Find customers whose names contain exactly 4 characters. (order results alphabetically)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium Q234. Find product names that are palindromes. (order results alphabetically)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q235. Return 'Yes' if an order is above \$500, else 'No'. (order results alphabetically)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q236. Update the stock quantity of a product after a sale. (order results alphabetically)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q237. Create a view that shows each customer's total order value. (order results alphabetically)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q238. Find columns with more than 5% NULL values (generic pattern). (order results alphabetically)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable  
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium

Q239. Find all managers who have more than 5 direct reports. (order results alphabetically)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY  
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard

Q240. Find customers who ordered the same product more than once. (order results alphabetically)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =  
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q241. Find the product with the most total units ordered. (order results alphabetically)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q242. Rank each order within a customer by amount, largest first. (order results alphabetically)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q243. Find products in the top 25% price within their category. (order results alphabetically)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q244. Find employees whose names contain two consecutive vowels. (order results alphabetically)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q245. Bucket orders into quartiles by amount using NTILE. (order results alphabetically)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q246. Archive old orders to an archive table and delete from main. (order results alphabetically)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard

Q247. Write a stored procedure to raise salary by a given percentage. (order results alphabetically)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q248. List all employees including those with no department, showing NULL as 'No Dept'. (limit to 10 results)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q249. Find employees who share the same salary as at least one other employee. (limit to 10 results)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q250. Find customers who ordered but live in countries with fewer than 3 total customers. (limit to 10 results)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q251. Find the total number of products per category. (limit to 10 results)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q252. Find categories with total revenue above \$20,000. (limit to 10 results)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q253. Find the department with the greatest salary variance. (limit to 10 results)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: `VARIANCE()` aggregate function.

Window Functions

Easy

Q254. Number each order within a customer partition by date. (limit to 10 results)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: `ROW_NUMBER` with `PARTITION BY`.

Medium

Q255. Find the running average salary as employees are scanned by hire date. (limit to 10 results)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running `AVG` with cumulative window.

Hard

Q256. Calculate the salary percentile rank for each employee. (limit to 10 results)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: `PERCENT_RANK()` returns 0 to 1.

CTEs & Subqueries

Easy

Q257. Using a CTE, list all customers who live in France. (limit to 10 results)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q258. Write a recursive CTE to show an employee org chart. (limit to 10 results)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: `RECURSIVE` CTE traverses a hierarchy.

Date & Time

Easy

Q259. Find orders placed today. (limit to 10 results)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: `DATE()` strips time part.

Medium

Q260. Find the busiest day of the week for orders. (limit to 10 results)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: `DAYNAME()` + `GROUP BY`.

Hard

Q261. Find customers who have not placed an order in over 6 months. (limit to 10 results)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q262. Find customers whose names contain exactly 4 characters. (limit to 10 results)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q263. Find product names that are palindromes. (limit to 10 results)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q264. Return 'Yes' if an order is above \$500, else 'No'. (limit to 10 results)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q265. Update the stock quantity of a product after a sale. (limit to 10 results)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q266. Create a view that shows each customer's total order value. (limit to 10 results)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q267. Find columns with more than 5% NULL values (generic pattern). (limit to 10 results)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q268. Find all managers who have more than 5 direct reports. (limit to 10 results)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q269. Find customers who ordered the same product more than once. (limit to 10 results)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q270. Find the product with the most total units ordered. (limit to 10 results)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q271. Rank each order within a customer by amount, largest first. (limit to 10 results)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q272. Find products in the top 25% price within their category. (limit to 10 results)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q273. Find employees whose names contain two consecutive vowels. (limit to 10 results)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q274. Bucket orders into quartiles by amount using NTILE. (limit to 10 results)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q275. Archive old orders to an archive table and delete from main. (limit to 10 results)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q276. Write a stored procedure to raise salary by a given percentage. (limit to 10 results)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q277. List all employees including those with no department, showing NULL as 'No Dept'. (use a CTE)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q278. Find employees who share the same salary as at least one other employee. (use a CTE)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q279. Find customers who ordered but live in countries with fewer than 3 total customers. (use a CTE)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q280. Find the total number of products per category. (use a CTE)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q281. Find categories with total revenue above \$20,000. (use a CTE)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q282. Find the department with the greatest salary variance. (use a CTE)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: `VARIANCE()` aggregate function.

Window Functions

Easy

Q283. Number each order within a customer partition by date. (use a CTE)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: `ROW_NUMBER` with `PARTITION BY`.

Medium

Q284. Find the running average salary as employees are scanned by hire date. (use a CTE)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running `AVG` with cumulative window.

Hard

Q285. Calculate the salary percentile rank for each employee. (use a CTE)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: `PERCENT_RANK()` returns 0 to 1.

CTEs & Subqueries

Easy

Q286. Using a CTE, list all customers who live in France. (use a CTE)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q287. Write a recursive CTE to show an employee org chart. (use a CTE)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: `RECURSIVE` CTE traverses a hierarchy.

Date & Time

Easy

Q288. Find orders placed today. (use a CTE)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: `DATE()` strips time part.

Medium

Q289. Find the busiest day of the week for orders. (use a CTE)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: `DAYNAME()` + `GROUP BY`.

Hard

Q290. Find customers who have not placed an order in over 6 months. (use a CTE)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q291. Find customers whose names contain exactly 4 characters. (use a CTE)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q292. Find product names that are palindromes. (use a CTE)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q293. Return 'Yes' if an order is above \$500, else 'No'. (use a CTE)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q294. Update the stock quantity of a product after a sale. (use a CTE)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q295. Create a view that shows each customer's total order value. (use a CTE)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q296. Find columns with more than 5% NULL values (generic pattern). (use a CTE)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium

Q297. Find all managers who have more than 5 direct reports. (use a CTE)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard

Q298. Find customers who ordered the same product more than once. (use a CTE)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium

Q299. Find the product with the most total units ordered. (use a CTE)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium

Q300. Rank each order within a customer by amount, largest first. (use a CTE)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard

Q301. Find products in the top 25% price within their category. (use a CTE)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium

Q302. Find employees whose names contain two consecutive vowels. (use a CTE)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium

Q303. Bucket orders into quartiles by amount using NTILE. (use a CTE)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q304. Archive old orders to an archive table and delete from main. (use a CTE)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q305. Write a stored procedure to raise salary by a given percentage. (use a CTE)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q306. List all employees including those with no department, showing NULL as 'No Dept'. (use a subquery instead)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q307. Find employees who share the same salary as at least one other employee. (use a subquery instead)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q308. Find customers who ordered but live in countries with fewer than 3 total customers. (use a subquery instead)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q309. Find the total number of products per category. (use a subquery instead)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q310. Find categories with total revenue above \$20,000. (use a subquery instead)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q311. Find the department with the greatest salary variance. (use a subquery instead)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q312. Number each order within a customer partition by date. (use a subquery instead)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q313. Find the running average salary as employees are scanned by hire date. (use a subquery instead)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q314. Calculate the salary percentile rank for each employee. (use a subquery instead)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q315. Using a CTE, list all customers who live in France. (use a subquery instead)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q316. Write a recursive CTE to show an employee org chart. (use a subquery instead)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q317. Find orders placed today. (use a subquery instead)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q318. Find the busiest day of the week for orders. (use a subquery instead)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q319. Find customers who have not placed an order in over 6 months. (use a subquery instead)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q320. Find customers whose names contain exactly 4 characters. (use a subquery instead)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q321. Find product names that are palindromes. (use a subquery instead)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q322. Return 'Yes' if an order is above \$500, else 'No'. (use a subquery instead)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q323. Update the stock quantity of a product after a sale. (use a subquery instead)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q324. Create a view that shows each customer's total order value. (use a subquery instead)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q325. Find columns with more than 5% NULL values (generic pattern). (use a subquery instead)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q326. Find all managers who have more than 5 direct reports. (use a subquery instead)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q327. Find customers who ordered the same product more than once. (use a subquery instead)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q328. Find the product with the most total units ordered. (use a subquery instead)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q329. Rank each order within a customer by amount, largest first. (use a subquery instead)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q330. Find products in the top 25% price within their category. (use a subquery instead)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q331. Find employees whose names contain two consecutive vowels. (use a subquery instead)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q332. Bucket orders into quartiles by amount using NTILE. (use a subquery instead)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q333. Archive old orders to an archive table and delete from main. (use a subquery instead)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q334. Write a stored procedure to raise salary by a given percentage. (use a subquery instead)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q335. List all employees including those with no department, showing NULL as 'No Dept'. (add an ORDER BY clause)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q336. Find employees who share the same salary as at least one other employee. (add an ORDER BY clause)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q337. Find customers who ordered but live in countries with fewer than 3 total customers. (add an ORDER BY clause)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q338. Find the total number of products per category. (add an ORDER BY clause)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q339. Find categories with total revenue above \$20,000. (add an ORDER BY clause)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q340. Find the department with the greatest salary variance. (add an ORDER BY clause)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q341. Number each order within a customer partition by date. (add an ORDER BY clause)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q342. Find the running average salary as employees are scanned by hire date. (add an ORDER BY clause)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard

Q343. Calculate the salary percentile rank for each employee. (add an ORDER BY clause)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q344. Using a CTE, list all customers who live in France. (add an ORDER BY clause)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q345. Write a recursive CTE to show an employee org chart. (add an ORDER BY clause)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q346. Find orders placed today. (add an ORDER BY clause)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q347. Find the busiest day of the week for orders. (add an ORDER BY clause)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q348. Find customers who have not placed an order in over 6 months. (add an ORDER BY clause)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q349. Find customers whose names contain exactly 4 characters. (add an ORDER BY clause)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q350. Find product names that are palindromes. (add an ORDER BY clause)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q351. Return 'Yes' if an order is above \$500, else 'No'. (add an ORDER BY clause)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q352. Update the stock quantity of a product after a sale. (add an ORDER BY clause)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q353. Create a view that shows each customer's total order value. (add an ORDER BY clause)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q354. Find columns with more than 5% NULL values (generic pattern). (add an ORDER BY clause)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q355. Find all managers who have more than 5 direct reports. (add an ORDER BY clause)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q356. Find customers who ordered the same product more than once. (add an ORDER BY clause)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q357. Find the product with the most total units ordered. (add an ORDER BY clause)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q358. Rank each order within a customer by amount, largest first. (add an ORDER BY clause)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q359. Find products in the top 25% price within their category. (add an ORDER BY clause)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM
Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q360. Find employees whose names contain two consecutive vowels. (add an ORDER BY clause)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q361. Bucket orders into quartiles by amount using NTILE. (add an ORDER BY clause)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q362. Archive old orders to an archive table and delete from main. (add an ORDER BY clause)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q363. Write a stored procedure to raise salary by a given percentage. (add an ORDER BY clause)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q364. List all employees including those with no department, showing NULL as 'No Dept'. (include NULL handling)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q365. Find employees who share the same salary as at least one other employee. (include NULL handling)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q366. Find customers who ordered but live in countries with fewer than 3 total customers. (include NULL handling)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q367. Find the total number of products per category. (include NULL handling)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q368. Find categories with total revenue above \$20,000. (include NULL handling)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id =
oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount)
> 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q369. Find the department with the greatest salary variance. (include NULL handling)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY
sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q370. Number each order within a customer partition by date. (include NULL handling)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q371. Find the running average salary as employees are scanned by hire date. (include NULL handling)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS
running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q372. Calculate the salary percentile rank for each employee. (include NULL handling)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q373. Using a CTE, list all customers who live in France. (include NULL handling)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q374. Write a recursive CTE to show an employee org chart. (include NULL handling)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q375. Find orders placed today. (include NULL handling)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q376. Find the busiest day of the week for orders. (include NULL handling)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q377. Find customers who have not placed an order in over 6 months. (include NULL handling)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q378. Find customers whose names contain exactly 4 characters. (include NULL handling)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q379. Find product names that are palindromes. (include NULL handling)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q380. Return 'Yes' if an order is above \$500, else 'No'. (include NULL handling)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q381. Update the stock quantity of a product after a sale. (include NULL handling)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```


Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q382. Create a view that shows each customer's total order value. (include NULL handling)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q383. Find columns with more than 5% NULL values (generic pattern). (include NULL handling)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q384. Find all managers who have more than 5 direct reports. (include NULL handling)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q385. Find customers who ordered the same product more than once. (include NULL handling)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q386. Find the product with the most total units ordered. (include NULL handling)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q387. Rank each order within a customer by amount, largest first. (include NULL handling)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q388. Find products in the top 25% price within their category. (include NULL handling)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q389. Find employees whose names contain two consecutive vowels. (include NULL handling)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q390. Bucket orders into quartiles by amount using NTILE. (include NULL handling)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q391. Archive old orders to an archive table and delete from main. (include NULL handling)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q392. Write a stored procedure to raise salary by a given percentage. (include NULL handling)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q393. List all employees including those with no department, showing NULL as 'No Dept'.

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q394. Find employees who share the same salary as at least one other employee.

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q395. Find customers who ordered but live in countries with fewer than 3 total customers.

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q396. Find the total number of products per category.

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q397. Find categories with total revenue above \$20,000.

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q398. Find the department with the greatest salary variance.

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q399. Number each order within a customer partition by date.

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q400. Find the running average salary as employees are scanned by hire date.

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q401. Calculate the salary percentile rank for each employee.

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q402. Using a CTE, list all customers who live in France.

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q403. Write a recursive CTE to show an employee org chart.

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q404. Find orders placed today.

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q405. Find the busiest day of the week for orders.

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard Q406. Find customers who have not placed an order in over 6 months.

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy Q407. Find customers whose names contain exactly 4 characters.

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium Q408. Find product names that are palindromes.

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy Q409. Return 'Yes' if an order is above \$500, else 'No'.

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium Q410. Update the stock quantity of a product after a sale.

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q411. Create a view that shows each customer's total order value.

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q412. Find columns with more than 5% NULL values (generic pattern).

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q413. Find all managers who have more than 5 direct reports.

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q414. Find customers who ordered the same product more than once.

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q415. Find the product with the most total units ordered.

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q416. Rank each order within a customer by amount, largest first.

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard

Q417. Find products in the top 25% price within their category.

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium

Q418. Find employees whose names contain two consecutive vowels.

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium

Q419. Bucket orders into quartiles by amount using NTILE.

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard

Q420. Archive old orders to an archive table and delete from main.

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard

Q421. Write a stored procedure to raise salary by a given percentage.

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q422. List all employees including those with no department, showing NULL as 'No Dept'. (include only active records)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q423. Find employees who share the same salary as at least one other employee. (include only active records)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q424. Find customers who ordered but live in countries with fewer than 3 total customers. (include only active records)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q425. Find the total number of products per category. (include only active records)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q426. Find categories with total revenue above \$20,000. (include only active records)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q427. Find the department with the greatest salary variance. (include only active records)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q428. Number each order within a customer partition by date. (include only active records)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q429. Find the running average salary as employees are scanned by hire date. (include only active records)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q430. Calculate the salary percentile rank for each employee. (include only active records)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q431. Using a CTE, list all customers who live in France. (include only active records)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q432. Write a recursive CTE to show an employee org chart. (include only active records)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q433. Find orders placed today. (include only active records)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q434. Find the busiest day of the week for orders. (include only active records)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q435. Find customers who have not placed an order in over 6 months. (include only active records)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q436. Find customers whose names contain exactly 4 characters. (include only active records)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q437. Find product names that are palindromes. (include only active records)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q438. Return 'Yes' if an order is above \$500, else 'No'. (include only active records)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium Q439. Update the stock quantity of a product after a sale. (include only active records)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q440. Create a view that shows each customer's total order value. (include only active records)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q441. Find columns with more than 5% NULL values (generic pattern). (include only active records)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable  
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q442. Find all managers who have more than 5 direct reports. (include only active records)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY  
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q443. Find customers who ordered the same product more than once. (include only active records)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =  
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q444. Find the product with the most total units ordered. (include only active records)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units  
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q445. Rank each order within a customer by amount, largest first. (include only active records)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q446. Find products in the top 25% price within their category. (include only active records)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q447. Find employees whose names contain two consecutive vowels. (include only active records)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q448. Bucket orders into quartiles by amount using NTILE. (include only active records)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q449. Archive old orders to an archive table and delete from main. (include only active records)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q450. Write a stored procedure to raise salary by a given percentage. (include only active records)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q451. List all employees including those with no department, showing NULL as 'No Dept'. (order results alphabetically)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q452. Find employees who share the same salary as at least one other employee. (order results alphabetically)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q453. Find customers who ordered but live in countries with fewer than 3 total customers. (order results alphabetically)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q454. Find the total number of products per category. (order results alphabetically)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q455. Find categories with total revenue above \$20,000. (order results alphabetically)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q456. Find the department with the greatest salary variance. (order results alphabetically)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q457. Number each order within a customer partition by date. (order results alphabetically)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q458. Find the running average salary as employees are scanned by hire date. (order results alphabetically)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q459. Calculate the salary percentile rank for each employee. (order results alphabetically)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q460. Using a CTE, list all customers who live in France. (order results alphabetically)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q461. Write a recursive CTE to show an employee org chart. (order results alphabetically)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q462. Find orders placed today. (order results alphabetically)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q463. Find the busiest day of the week for orders. (order results alphabetically)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard Q464. Find customers who have not placed an order in over 6 months. (order results alphabetically)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy Q465. Find customers whose names contain exactly 4 characters. (order results alphabetically)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium Q466. Find product names that are palindromes. (order results alphabetically)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q467. Return 'Yes' if an order is above \$500, else 'No'. (order results alphabetically)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q468. Update the stock quantity of a product after a sale. (order results alphabetically)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q469. Create a view that shows each customer's total order value. (order results alphabetically)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q470. Find columns with more than 5% NULL values (generic pattern). (order results alphabetically)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable  
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium

Q471. Find all managers who have more than 5 direct reports. (order results alphabetically)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY  
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard

Q472. Find customers who ordered the same product more than once. (order results alphabetically)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =  
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q473. Find the product with the most total units ordered. (order results alphabetically)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q474. Rank each order within a customer by amount, largest first. (order results alphabetically)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q475. Find products in the top 25% price within their category. (order results alphabetically)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q476. Find employees whose names contain two consecutive vowels. (order results alphabetically)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q477. Bucket orders into quartiles by amount using NTILE. (order results alphabetically)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q478. Archive old orders to an archive table and delete from main. (order results alphabetically)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard

Q479. Write a stored procedure to raise salary by a given percentage. (order results alphabetically)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q480. List all employees including those with no department, showing NULL as 'No Dept'. (limit to 10 results)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q481. Find employees who share the same salary as at least one other employee. (limit to 10 results)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q482. Find customers who ordered but live in countries with fewer than 3 total customers. (limit to 10 results)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q483. Find the total number of products per category. (limit to 10 results)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q484. Find categories with total revenue above \$20,000. (limit to 10 results)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q485. Find the department with the greatest salary variance. (limit to 10 results)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: `VARIANCE()` aggregate function.

Window Functions

Easy Q486. Number each order within a customer partition by date. (limit to 10 results)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: `ROW_NUMBER` with `PARTITION BY`.

Medium Q487. Find the running average salary as employees are scanned by hire date. (limit to 10 results)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running `AVG` with cumulative window.

Hard Q488. Calculate the salary percentile rank for each employee. (limit to 10 results)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: `PERCENT_RANK()` returns 0 to 1.

CTEs & Subqueries

Easy Q489. Using a CTE, list all customers who live in France. (limit to 10 results)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q490. Write a recursive CTE to show an employee org chart. (limit to 10 results)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: `RECURSIVE` CTE traverses a hierarchy.

Date & Time

Easy Q491. Find orders placed today. (limit to 10 results)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: `DATE()` strips time part.

Medium Q492. Find the busiest day of the week for orders. (limit to 10 results)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: `DAYNAME()` + `GROUP BY`.

Hard Q493. Find customers who have not placed an order in over 6 months. (limit to 10 results)


```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q494. Find customers whose names contain exactly 4 characters. (limit to 10 results)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q495. Find product names that are palindromes. (limit to 10 results)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q496. Return 'Yes' if an order is above \$500, else 'No'. (limit to 10 results)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q497. Update the stock quantity of a product after a sale. (limit to 10 results)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q498. Create a view that shows each customer's total order value. (limit to 10 results)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q499. Find columns with more than 5% NULL values (generic pattern). (limit to 10 results)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q500. Find all managers who have more than 5 direct reports. (limit to 10 results)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q501. Find customers who ordered the same product more than once. (limit to 10 results)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q502. Find the product with the most total units ordered. (limit to 10 results)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q503. Rank each order within a customer by amount, largest first. (limit to 10 results)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q504. Find products in the top 25% price within their category. (limit to 10 results)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q505. Find employees whose names contain two consecutive vowels. (limit to 10 results)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q506. Bucket orders into quartiles by amount using NTILE. (limit to 10 results)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q507. Archive old orders to an archive table and delete from main. (limit to 10 results)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q508. Write a stored procedure to raise salary by a given percentage. (limit to 10 results)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q509. List all employees including those with no department, showing NULL as 'No Dept'. (use a CTE)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q510. Find employees who share the same salary as at least one other employee. (use a CTE)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q511. Find customers who ordered but live in countries with fewer than 3 total customers. (use a CTE)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q512. Find the total number of products per category. (use a CTE)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q513. Find categories with total revenue above \$20,000. (use a CTE)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q514. Find the department with the greatest salary variance. (use a CTE)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: `VARIANCE()` aggregate function.

Window Functions

Easy

Q515. Number each order within a customer partition by date. (use a CTE)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: `ROW_NUMBER` with `PARTITION BY`.

Medium

Q516. Find the running average salary as employees are scanned by hire date. (use a CTE)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running `AVG` with cumulative window.

Hard

Q517. Calculate the salary percentile rank for each employee. (use a CTE)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: `PERCENT_RANK()` returns 0 to 1.

CTEs & Subqueries

Easy

Q518. Using a CTE, list all customers who live in France. (use a CTE)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q519. Write a recursive CTE to show an employee org chart. (use a CTE)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: `RECURSIVE` CTE traverses a hierarchy.

Date & Time

Easy

Q520. Find orders placed today. (use a CTE)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: `DATE()` strips time part.

Medium

Q521. Find the busiest day of the week for orders. (use a CTE)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: `DAYNAME()` + `GROUP BY`.

Hard

Q522. Find customers who have not placed an order in over 6 months. (use a CTE)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q523. Find customers whose names contain exactly 4 characters. (use a CTE)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q524. Find product names that are palindromes. (use a CTE)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q525. Return 'Yes' if an order is above \$500, else 'No'. (use a CTE)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q526. Update the stock quantity of a product after a sale. (use a CTE)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q527. Create a view that shows each customer's total order value. (use a CTE)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q528. Find columns with more than 5% NULL values (generic pattern). (use a CTE)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q529. Find all managers who have more than 5 direct reports. (use a CTE)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q530. Find customers who ordered the same product more than once. (use a CTE)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q531. Find the product with the most total units ordered. (use a CTE)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q532. Rank each order within a customer by amount, largest first. (use a CTE)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q533. Find products in the top 25% price within their category. (use a CTE)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q534. Find employees whose names contain two consecutive vowels. (use a CTE)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q535. Bucket orders into quartiles by amount using NTILE. (use a CTE)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q536. Archive old orders to an archive table and delete from main. (use a CTE)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q537. Write a stored procedure to raise salary by a given percentage. (use a CTE)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q538. List all employees including those with no department, showing NULL as 'No Dept'. (use a subquery instead)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q539. Find employees who share the same salary as at least one other employee. (use a subquery instead)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q540. Find customers who ordered but live in countries with fewer than 3 total customers. (use a subquery instead)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q541. Find the total number of products per category. (use a subquery instead)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q542. Find categories with total revenue above \$20,000. (use a subquery instead)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q543. Find the department with the greatest salary variance. (use a subquery instead)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q544. Number each order within a customer partition by date. (use a subquery instead)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q545. Find the running average salary as employees are scanned by hire date. (use a subquery instead)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q546. Calculate the salary percentile rank for each employee. (use a subquery instead)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q547. Using a CTE, list all customers who live in France. (use a subquery instead)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q548. Write a recursive CTE to show an employee org chart. (use a subquery instead)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q549. Find orders placed today. (use a subquery instead)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q550. Find the busiest day of the week for orders. (use a subquery instead)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q551. Find customers who have not placed an order in over 6 months. (use a subquery instead)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q552. Find customers whose names contain exactly 4 characters. (use a subquery instead)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q553. Find product names that are palindromes. (use a subquery instead)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q554. Return 'Yes' if an order is above \$500, else 'No'. (use a subquery instead)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q555. Update the stock quantity of a product after a sale. (use a subquery instead)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q556. Create a view that shows each customer's total order value. (use a subquery instead)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q557. Find columns with more than 5% NULL values (generic pattern). (use a subquery instead)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q558. Find all managers who have more than 5 direct reports. (use a subquery instead)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q559. Find customers who ordered the same product more than once. (use a subquery instead)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q560. Find the product with the most total units ordered. (use a subquery instead)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q561. Rank each order within a customer by amount, largest first. (use a subquery instead)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q562. Find products in the top 25% price within their category. (use a subquery instead)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q563. Find employees whose names contain two consecutive vowels. (use a subquery instead)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q564. Bucket orders into quartiles by amount using NTILE. (use a subquery instead)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q565. Archive old orders to an archive table and delete from main. (use a subquery instead)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q566. Write a stored procedure to raise salary by a given percentage. (use a subquery instead)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q567. List all employees including those with no department, showing NULL as 'No Dept'. (add an ORDER BY clause)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q568. Find employees who share the same salary as at least one other employee. (add an ORDER BY clause)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q569. Find customers who ordered but live in countries with fewer than 3 total customers. (add an ORDER BY clause)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q570. Find the total number of products per category. (add an ORDER BY clause)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q571. Find categories with total revenue above \$20,000. (add an ORDER BY clause)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q572. Find the department with the greatest salary variance. (add an ORDER BY clause)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q573. Number each order within a customer partition by date. (add an ORDER BY clause)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q574. Find the running average salary as employees are scanned by hire date. (add an ORDER BY clause)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard

Q575. Calculate the salary percentile rank for each employee. (add an ORDER BY clause)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q576. Using a CTE, list all customers who live in France. (add an ORDER BY clause)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q577. Write a recursive CTE to show an employee org chart. (add an ORDER BY clause)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q578. Find orders placed today. (add an ORDER BY clause)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q579. Find the busiest day of the week for orders. (add an ORDER BY clause)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q580. Find customers who have not placed an order in over 6 months. (add an ORDER BY clause)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q581. Find customers whose names contain exactly 4 characters. (add an ORDER BY clause)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q582. Find product names that are palindromes. (add an ORDER BY clause)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q583. Return 'Yes' if an order is above \$500, else 'No'. (add an ORDER BY clause)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q584. Update the stock quantity of a product after a sale. (add an ORDER BY clause)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q585. Create a view that shows each customer's total order value. (add an ORDER BY clause)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q586. Find columns with more than 5% NULL values (generic pattern). (add an ORDER BY clause)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q587. Find all managers who have more than 5 direct reports. (add an ORDER BY clause)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q588. Find customers who ordered the same product more than once. (add an ORDER BY clause)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q589. Find the product with the most total units ordered. (add an ORDER BY clause)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q590. Rank each order within a customer by amount, largest first. (add an ORDER BY clause)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q591. Find products in the top 25% price within their category. (add an ORDER BY clause)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM
Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q592. Find employees whose names contain two consecutive vowels. (add an ORDER BY clause)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q593. Bucket orders into quartiles by amount using NTILE. (add an ORDER BY clause)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q594. Archive old orders to an archive table and delete from main. (add an ORDER BY clause)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q595. Write a stored procedure to raise salary by a given percentage. (add an ORDER BY clause)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q596. List all employees including those with no department, showing NULL as 'No Dept'. (include NULL handling)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q597. Find employees who share the same salary as at least one other employee. (include NULL handling)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q598. Find customers who ordered but live in countries with fewer than 3 total customers. (include NULL handling)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q599. Find the total number of products per category. (include NULL handling)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q600. Find categories with total revenue above \$20,000. (include NULL handling)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id =
oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount)
> 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q601. Find the department with the greatest salary variance. (include NULL handling)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY
sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q602. Number each order within a customer partition by date. (include NULL handling)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q603. Find the running average salary as employees are scanned by hire date. (include NULL handling)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS
running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard

Q604. Calculate the salary percentile rank for each employee. (include NULL handling)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q605. Using a CTE, list all customers who live in France. (include NULL handling)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q606. Write a recursive CTE to show an employee org chart. (include NULL handling)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q607. Find orders placed today. (include NULL handling)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q608. Find the busiest day of the week for orders. (include NULL handling)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q609. Find customers who have not placed an order in over 6 months. (include NULL handling)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q610. Find customers whose names contain exactly 4 characters. (include NULL handling)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q611. Find product names that are palindromes. (include NULL handling)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q612. Return 'Yes' if an order is above \$500, else 'No'. (include NULL handling)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q613. Update the stock quantity of a product after a sale. (include NULL handling)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q614. Create a view that shows each customer's total order value. (include NULL handling)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q615. Find columns with more than 5% NULL values (generic pattern). (include NULL handling)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q616. Find all managers who have more than 5 direct reports. (include NULL handling)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q617. Find customers who ordered the same product more than once. (include NULL handling)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q618. Find the product with the most total units ordered. (include NULL handling)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q619. Rank each order within a customer by amount, largest first. (include NULL handling)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q620. Find products in the top 25% price within their category. (include NULL handling)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q621. Find employees whose names contain two consecutive vowels. (include NULL handling)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q622. Bucket orders into quartiles by amount using NTILE. (include NULL handling)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q623. Archive old orders to an archive table and delete from main. (include NULL handling)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q624. Write a stored procedure to raise salary by a given percentage. (include NULL handling)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q625. List all employees including those with no department, showing NULL as 'No Dept'.

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q626. Find employees who share the same salary as at least one other employee.

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q627. Find customers who ordered but live in countries with fewer than 3 total customers.

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q628. Find the total number of products per category.

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q629. Find categories with total revenue above \$20,000.

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q630. Find the department with the greatest salary variance.

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q631. Number each order within a customer partition by date.

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q632. Find the running average salary as employees are scanned by hire date.

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q633. Calculate the salary percentile rank for each employee.

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q634. Using a CTE, list all customers who live in France.

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q635. Write a recursive CTE to show an employee org chart.

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q636. Find orders placed today.

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q637. Find the busiest day of the week for orders.

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q638. Find customers who have not placed an order in over 6 months.

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q639. Find customers whose names contain exactly 4 characters.

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q640. Find product names that are palindromes.

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q641. Return 'Yes' if an order is above \$500, else 'No'.

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q642. Update the stock quantity of a product after a sale.

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q643. Create a view that shows each customer's total order value.

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q644. Find columns with more than 5% NULL values (generic pattern).

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q645. Find all managers who have more than 5 direct reports.

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q646. Find customers who ordered the same product more than once.

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q647. Find the product with the most total units ordered.

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q648. Rank each order within a customer by amount, largest first.

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard

Q649. Find products in the top 25% price within their category.

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: `NTILE(4)` then filter quartile 1.

String Functions

Medium

Q650. Find employees whose names contain two consecutive vowels.

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: `REGEXP` with repetition quantifier.

CASE & Logic

Medium

Q651. Bucket orders into quartiles by amount using `NTILE`.

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: `NTILE(4)` on amount.

DML (INSERT/UPDATE/DELETE)

Hard

Q652. Archive old orders to an archive table and delete from main.

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: `INSERT...SELECT` then `DELETE`.

Schema & Indexes

Hard

Q653. Write a stored procedure to raise salary by a given percentage.

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with `IN` parameter.

JOINS

Easy

Q654. List all employees including those with no department, showing `NULL` as 'No Dept'. (include only active records)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: `LEFT JOIN` + `COALESCE` for `NULL` dept.

Medium

Q655. Find employees who share the same salary as at least one other employee. (include only active records)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q656. Find customers who ordered but live in countries with fewer than 3 total customers. (include only active records)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q657. Find the total number of products per category. (include only active records)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q658. Find categories with total revenue above \$20,000. (include only active records)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q659. Find the department with the greatest salary variance. (include only active records)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q660. Number each order within a customer partition by date. (include only active records)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q661. Find the running average salary as employees are scanned by hire date. (include only active records)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q662. Calculate the salary percentile rank for each employee. (include only active records)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q663. Using a CTE, list all customers who live in France. (include only active records)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q664. Write a recursive CTE to show an employee org chart. (include only active records)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q665. Find orders placed today. (include only active records)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q666. Find the busiest day of the week for orders. (include only active records)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q667. Find customers who have not placed an order in over 6 months. (include only active records)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q668. Find customers whose names contain exactly 4 characters. (include only active records)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q669. Find product names that are palindromes. (include only active records)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q670. Return 'Yes' if an order is above \$500, else 'No'. (include only active records)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium Q671. Update the stock quantity of a product after a sale. (include only active records)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q672. Create a view that shows each customer's total order value. (include only active records)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q673. Find columns with more than 5% NULL values (generic pattern). (include only active records)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable  
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q674. Find all managers who have more than 5 direct reports. (include only active records)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY  
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q675. Find customers who ordered the same product more than once. (include only active records)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =  
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q676. Find the product with the most total units ordered. (include only active records)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units  
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q677. Rank each order within a customer by amount, largest first. (include only active records)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q678. Find products in the top 25% price within their category. (include only active records)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q679. Find employees whose names contain two consecutive vowels. (include only active records)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q680. Bucket orders into quartiles by amount using NTILE. (include only active records)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q681. Archive old orders to an archive table and delete from main. (include only active records)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q682. Write a stored procedure to raise salary by a given percentage. (include only active records)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q683. List all employees including those with no department, showing NULL as 'No Dept'. (order results alphabetically)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q684. Find employees who share the same salary as at least one other employee. (order results alphabetically)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q685. Find customers who ordered but live in countries with fewer than 3 total customers. (order results alphabetically)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q686. Find the total number of products per category. (order results alphabetically)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q687. Find categories with total revenue above \$20,000. (order results alphabetically)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q688. Find the department with the greatest salary variance. (order results alphabetically)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q689. Number each order within a customer partition by date. (order results alphabetically)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q690. Find the running average salary as employees are scanned by hire date. (order results alphabetically)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q691. Calculate the salary percentile rank for each employee. (order results alphabetically)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q692. Using a CTE, list all customers who live in France. (order results alphabetically)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q693. Write a recursive CTE to show an employee org chart. (order results alphabetically)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q694. Find orders placed today. (order results alphabetically)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q695. Find the busiest day of the week for orders. (order results alphabetically)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard Q696. Find customers who have not placed an order in over 6 months. (order results alphabetically)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy Q697. Find customers whose names contain exactly 4 characters. (order results alphabetically)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium Q698. Find product names that are palindromes. (order results alphabetically)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy Q699. Return 'Yes' if an order is above \$500, else 'No'. (order results alphabetically)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium Q700. Update the stock quantity of a product after a sale. (order results alphabetically)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q701. Create a view that shows each customer's total order value. (order results alphabetically)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q702. Find columns with more than 5% NULL values (generic pattern). (order results alphabetically)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable  
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q703. Find all managers who have more than 5 direct reports. (order results alphabetically)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY  
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q704. Find customers who ordered the same product more than once. (order results alphabetically)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =  
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q705. Find the product with the most total units ordered. (order results alphabetically)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q706. Rank each order within a customer by amount, largest first. (order results alphabetically)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q707. Find products in the top 25% price within their category. (order results alphabetically)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q708. Find employees whose names contain two consecutive vowels. (order results alphabetically)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q709. Bucket orders into quartiles by amount using NTILE. (order results alphabetically)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q710. Archive old orders to an archive table and delete from main. (order results alphabetically)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard

Q711. Write a stored procedure to raise salary by a given percentage. (order results alphabetically)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q712. List all employees including those with no department, showing NULL as 'No Dept'. (limit to 10 results)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q713. Find employees who share the same salary as at least one other employee. (limit to 10 results)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q714. Find customers who ordered but live in countries with fewer than 3 total customers. (limit to 10 results)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q715. Find the total number of products per category. (limit to 10 results)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q716. Find categories with total revenue above \$20,000. (limit to 10 results)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q717. Find the department with the greatest salary variance. (limit to 10 results)


```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: `VARIANCE()` aggregate function.

Window Functions

Easy

Q718. Number each order within a customer partition by date. (limit to 10 results)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: `ROW_NUMBER` with `PARTITION BY`.

Medium

Q719. Find the running average salary as employees are scanned by hire date. (limit to 10 results)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running `AVG` with cumulative window.

Hard

Q720. Calculate the salary percentile rank for each employee. (limit to 10 results)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: `PERCENT_RANK()` returns 0 to 1.

CTEs & Subqueries

Easy

Q721. Using a CTE, list all customers who live in France. (limit to 10 results)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q722. Write a recursive CTE to show an employee org chart. (limit to 10 results)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: `RECURSIVE` CTE traverses a hierarchy.

Date & Time

Easy

Q723. Find orders placed today. (limit to 10 results)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: `DATE()` strips time part.

Medium

Q724. Find the busiest day of the week for orders. (limit to 10 results)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: `DAYNAME()` + `GROUP BY`.

Hard

Q725. Find customers who have not placed an order in over 6 months. (limit to 10 results)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q726. Find customers whose names contain exactly 4 characters. (limit to 10 results)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q727. Find product names that are palindromes. (limit to 10 results)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q728. Return 'Yes' if an order is above \$500, else 'No'. (limit to 10 results)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q729. Update the stock quantity of a product after a sale. (limit to 10 results)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q730. Create a view that shows each customer's total order value. (limit to 10 results)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q731. Find columns with more than 5% NULL values (generic pattern). (limit to 10 results)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium

Q732. Find all managers who have more than 5 direct reports. (limit to 10 results)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard

Q733. Find customers who ordered the same product more than once. (limit to 10 results)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium

Q734. Find the product with the most total units ordered. (limit to 10 results)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium

Q735. Rank each order within a customer by amount, largest first. (limit to 10 results)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard

Q736. Find products in the top 25% price within their category. (limit to 10 results)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium

Q737. Find employees whose names contain two consecutive vowels. (limit to 10 results)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium

Q738. Bucket orders into quartiles by amount using NTILE. (limit to 10 results)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q739. Archive old orders to an archive table and delete from main. (limit to 10 results)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q740. Write a stored procedure to raise salary by a given percentage. (limit to 10 results)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q741. List all employees including those with no department, showing NULL as 'No Dept'. (use a CTE)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q742. Find employees who share the same salary as at least one other employee. (use a CTE)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q743. Find customers who ordered but live in countries with fewer than 3 total customers. (use a CTE)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q744. Find the total number of products per category. (use a CTE)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q745. Find categories with total revenue above \$20,000. (use a CTE)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q746. Find the department with the greatest salary variance. (use a CTE)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: `VARIANCE()` aggregate function.

Window Functions

Easy

Q747. Number each order within a customer partition by date. (use a CTE)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: `ROW_NUMBER` with `PARTITION BY`.

Medium

Q748. Find the running average salary as employees are scanned by hire date. (use a CTE)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running `AVG` with cumulative window.

Hard

Q749. Calculate the salary percentile rank for each employee. (use a CTE)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: `PERCENT_RANK()` returns 0 to 1.

CTEs & Subqueries

Easy

Q750. Using a CTE, list all customers who live in France. (use a CTE)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q751. Write a recursive CTE to show an employee org chart. (use a CTE)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: `RECURSIVE` CTE traverses a hierarchy.

Date & Time

Easy

Q752. Find orders placed today. (use a CTE)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: `DATE()` strips time part.

Medium

Q753. Find the busiest day of the week for orders. (use a CTE)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: `DAYNAME()` + `GROUP BY`.

Hard

Q754. Find customers who have not placed an order in over 6 months. (use a CTE)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q755. Find customers whose names contain exactly 4 characters. (use a CTE)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q756. Find product names that are palindromes. (use a CTE)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q757. Return 'Yes' if an order is above \$500, else 'No'. (use a CTE)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q758. Update the stock quantity of a product after a sale. (use a CTE)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q759. Create a view that shows each customer's total order value. (use a CTE)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q760. Find columns with more than 5% NULL values (generic pattern). (use a CTE)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q761. Find all managers who have more than 5 direct reports. (use a CTE)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q762. Find customers who ordered the same product more than once. (use a CTE)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q763. Find the product with the most total units ordered. (use a CTE)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q764. Rank each order within a customer by amount, largest first. (use a CTE)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q765. Find products in the top 25% price within their category. (use a CTE)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q766. Find employees whose names contain two consecutive vowels. (use a CTE)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q767. Bucket orders into quartiles by amount using NTILE. (use a CTE)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q768. Archive old orders to an archive table and delete from main. (use a CTE)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q769. Write a stored procedure to raise salary by a given percentage. (use a CTE)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q770. List all employees including those with no department, showing NULL as 'No Dept'. (use a subquery instead)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q771. Find employees who share the same salary as at least one other employee. (use a subquery instead)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q772. Find customers who ordered but live in countries with fewer than 3 total customers. (use a subquery instead)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q773. Find the total number of products per category. (use a subquery instead)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q774. Find categories with total revenue above \$20,000. (use a subquery instead)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```


Tip: Multi-join + HAVING on SUM.

Hard Q775. Find the department with the greatest salary variance. (use a subquery instead)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q776. Number each order within a customer partition by date. (use a subquery instead)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q777. Find the running average salary as employees are scanned by hire date. (use a subquery instead)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q778. Calculate the salary percentile rank for each employee. (use a subquery instead)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q779. Using a CTE, list all customers who live in France. (use a subquery instead)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q780. Write a recursive CTE to show an employee org chart. (use a subquery instead)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q781. Find orders placed today. (use a subquery instead)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q782. Find the busiest day of the week for orders. (use a subquery instead)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q783. Find customers who have not placed an order in over 6 months. (use a subquery instead)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q784. Find customers whose names contain exactly 4 characters. (use a subquery instead)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q785. Find product names that are palindromes. (use a subquery instead)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q786. Return 'Yes' if an order is above \$500, else 'No'. (use a subquery instead)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q787. Update the stock quantity of a product after a sale. (use a subquery instead)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q788. Create a view that shows each customer's total order value. (use a subquery instead)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q789. Find columns with more than 5% NULL values (generic pattern). (use a subquery instead)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q790. Find all managers who have more than 5 direct reports. (use a subquery instead)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q791. Find customers who ordered the same product more than once. (use a subquery instead)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q792. Find the product with the most total units ordered. (use a subquery instead)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q793. Rank each order within a customer by amount, largest first. (use a subquery instead)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q794. Find products in the top 25% price within their category. (use a subquery instead)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q795. Find employees whose names contain two consecutive vowels. (use a subquery instead)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q796. Bucket orders into quartiles by amount using NTILE. (use a subquery instead)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q797. Archive old orders to an archive table and delete from main. (use a subquery instead)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q798. Write a stored procedure to raise salary by a given percentage. (use a subquery instead)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q799. List all employees including those with no department, showing NULL as 'No Dept'. (add an ORDER BY clause)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q800. Find employees who share the same salary as at least one other employee. (add an ORDER BY clause)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q801. Find customers who ordered but live in countries with fewer than 3 total customers. (add an ORDER BY clause)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q802. Find the total number of products per category. (add an ORDER BY clause)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q803. Find categories with total revenue above \$20,000. (add an ORDER BY clause)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q804. Find the department with the greatest salary variance. (add an ORDER BY clause)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q805. Number each order within a customer partition by date. (add an ORDER BY clause)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q806. Find the running average salary as employees are scanned by hire date. (add an ORDER BY clause)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard

Q807. Calculate the salary percentile rank for each employee. (add an ORDER BY clause)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q808. Using a CTE, list all customers who live in France. (add an ORDER BY clause)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q809. Write a recursive CTE to show an employee org chart. (add an ORDER BY clause)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q810. Find orders placed today. (add an ORDER BY clause)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q811. Find the busiest day of the week for orders. (add an ORDER BY clause)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q812. Find customers who have not placed an order in over 6 months. (add an ORDER BY clause)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q813. Find customers whose names contain exactly 4 characters. (add an ORDER BY clause)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q814. Find product names that are palindromes. (add an ORDER BY clause)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q815. Return 'Yes' if an order is above \$500, else 'No'. (add an ORDER BY clause)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q816. Update the stock quantity of a product after a sale. (add an ORDER BY clause)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q817. Create a view that shows each customer's total order value. (add an ORDER BY clause)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q818. Find columns with more than 5% NULL values (generic pattern). (add an ORDER BY clause)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q819. Find all managers who have more than 5 direct reports. (add an ORDER BY clause)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q820. Find customers who ordered the same product more than once. (add an ORDER BY clause)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q821. Find the product with the most total units ordered. (add an ORDER BY clause)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q822. Rank each order within a customer by amount, largest first. (add an ORDER BY clause)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q823. Find products in the top 25% price within their category. (add an ORDER BY clause)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM
Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q824. Find employees whose names contain two consecutive vowels. (add an ORDER BY clause)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q825. Bucket orders into quartiles by amount using NTILE. (add an ORDER BY clause)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q826. Archive old orders to an archive table and delete from main. (add an ORDER BY clause)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q827. Write a stored procedure to raise salary by a given percentage. (add an ORDER BY clause)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q828. List all employees including those with no department, showing NULL as 'No Dept'. (include NULL handling)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q829. Find employees who share the same salary as at least one other employee. (include NULL handling)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q830. Find customers who ordered but live in countries with fewer than 3 total customers. (include NULL handling)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q831. Find the total number of products per category. (include NULL handling)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q832. Find categories with total revenue above \$20,000. (include NULL handling)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id =
oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount)
> 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q833. Find the department with the greatest salary variance. (include NULL handling)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY
sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q834. Number each order within a customer partition by date. (include NULL handling)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q835. Find the running average salary as employees are scanned by hire date. (include NULL handling)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS
running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard

Q836. Calculate the salary percentile rank for each employee. (include NULL handling)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q837. Using a CTE, list all customers who live in France. (include NULL handling)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q838. Write a recursive CTE to show an employee org chart. (include NULL handling)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q839. Find orders placed today. (include NULL handling)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q840. Find the busiest day of the week for orders. (include NULL handling)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q841. Find customers who have not placed an order in over 6 months. (include NULL handling)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q842. Find customers whose names contain exactly 4 characters. (include NULL handling)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q843. Find product names that are palindromes. (include NULL handling)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q844. Return 'Yes' if an order is above \$500, else 'No'. (include NULL handling)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q845. Update the stock quantity of a product after a sale. (include NULL handling)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q846. Create a view that shows each customer's total order value. (include NULL handling)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q847. Find columns with more than 5% NULL values (generic pattern). (include NULL handling)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q848. Find all managers who have more than 5 direct reports. (include NULL handling)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q849. Find customers who ordered the same product more than once. (include NULL handling)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q850. Find the product with the most total units ordered. (include NULL handling)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q851. Rank each order within a customer by amount, largest first. (include NULL handling)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q852. Find products in the top 25% price within their category. (include NULL handling)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q853. Find employees whose names contain two consecutive vowels. (include NULL handling)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q854. Bucket orders into quartiles by amount using NTILE. (include NULL handling)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q855. Archive old orders to an archive table and delete from main. (include NULL handling)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q856. Write a stored procedure to raise salary by a given percentage. (include NULL handling)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q857. List all employees including those with no department, showing NULL as 'No Dept'.

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q858. Find employees who share the same salary as at least one other employee.

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q859. Find customers who ordered but live in countries with fewer than 3 total customers.

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q860. Find the total number of products per category.

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q861. Find categories with total revenue above \$20,000.

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q862. Find the department with the greatest salary variance.

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q863. Number each order within a customer partition by date.

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q864. Find the running average salary as employees are scanned by hire date.

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q865. Calculate the salary percentile rank for each employee.

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q866. Using a CTE, list all customers who live in France.

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q867. Write a recursive CTE to show an employee org chart.

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q868. Find orders placed today.

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q869. Find the busiest day of the week for orders.

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q870. Find customers who have not placed an order in over 6 months.

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q871. Find customers whose names contain exactly 4 characters.

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q872. Find product names that are palindromes.

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q873. Return 'Yes' if an order is above \$500, else 'No'.

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q874. Update the stock quantity of a product after a sale.

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q875. Create a view that shows each customer's total order value.

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q876. Find columns with more than 5% NULL values (generic pattern).

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q877. Find all managers who have more than 5 direct reports.

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q878. Find customers who ordered the same product more than once.

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q879. Find the product with the most total units ordered.

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q880. Rank each order within a customer by amount, largest first.

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard

Q881. Find products in the top 25% price within their category.

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: `NTILE(4)` then filter `quartile 1`.

String Functions

Medium

Q882. Find employees whose names contain two consecutive vowels.

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: `REGEXP` with repetition quantifier.

CASE & Logic

Medium

Q883. Bucket orders into quartiles by amount using `NTILE`.

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: `NTILE(4)` on amount.

DML (INSERT/UPDATE/DELETE)

Hard

Q884. Archive old orders to an archive table and delete from main.

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: `INSERT...SELECT` then `DELETE`.

Schema & Indexes

Hard

Q885. Write a stored procedure to raise salary by a given percentage.

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with `IN` parameter.

JOINS

Easy

Q886. List all employees including those with no department, showing `NULL` as 'No Dept'. (include only active records)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: `LEFT JOIN` + `COALESCE` for `NULL` dept.

Medium

Q887. Find employees who share the same salary as at least one other employee. (include only active records)


```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q888. Find customers who ordered but live in countries with fewer than 3 total customers. (include only active records)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q889. Find the total number of products per category. (include only active records)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q890. Find categories with total revenue above \$20,000. (include only active records)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard Q891. Find the department with the greatest salary variance. (include only active records)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy Q892. Number each order within a customer partition by date. (include only active records)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium Q893. Find the running average salary as employees are scanned by hire date. (include only active records)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q894. Calculate the salary percentile rank for each employee. (include only active records)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy

Q895. Using a CTE, list all customers who live in France. (include only active records)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q896. Write a recursive CTE to show an employee org chart. (include only active records)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy

Q897. Find orders placed today. (include only active records)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium

Q898. Find the busiest day of the week for orders. (include only active records)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard

Q899. Find customers who have not placed an order in over 6 months. (include only active records)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q900. Find customers whose names contain exactly 4 characters. (include only active records)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q901. Find product names that are palindromes. (include only active records)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q902. Return 'Yes' if an order is above \$500, else 'No'. (include only active records)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium Q903. Update the stock quantity of a product after a sale. (include only active records)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium Q904. Create a view that shows each customer's total order value. (include only active records)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium Q905. Find columns with more than 5% NULL values (generic pattern). (include only active records)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable  
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q906. Find all managers who have more than 5 direct reports. (include only active records)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY  
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q907. Find customers who ordered the same product more than once. (include only active records)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =  
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q908. Find the product with the most total units ordered. (include only active records)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units  
DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q909. Rank each order within a customer by amount, largest first. (include only active records)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q910. Find products in the top 25% price within their category. (include only active records)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q911. Find employees whose names contain two consecutive vowels. (include only active records)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q912. Bucket orders into quartiles by amount using NTILE. (include only active records)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q913. Archive old orders to an archive table and delete from main. (include only active records)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q914. Write a stored procedure to raise salary by a given percentage. (include only active records)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q915. List all employees including those with no department, showing NULL as 'No Dept'. (order results alphabetically)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q916. Find employees who share the same salary as at least one other employee. (order results alphabetically)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q917. Find customers who ordered but live in countries with fewer than 3 total customers. (order results alphabetically)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q918. Find the total number of products per category. (order results alphabetically)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q919. Find categories with total revenue above \$20,000. (order results alphabetically)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q920. Find the department with the greatest salary variance. (order results alphabetically)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: VARIANCE() aggregate function.

Window Functions

Easy

Q921. Number each order within a customer partition by date. (order results alphabetically)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: ROW_NUMBER with PARTITION BY.

Medium

Q922. Find the running average salary as employees are scanned by hire date. (order results alphabetically)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS
running_avg FROM Employees;
```

Tip: Running AVG with cumulative window.

Hard Q923. Calculate the salary percentile rank for each employee. (order results alphabetically)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: PERCENT_RANK() returns 0 to 1.

CTEs & Subqueries

Easy Q924. Using a CTE, list all customers who live in France. (order results alphabetically)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard Q925. Write a recursive CTE to show an employee org chart. (order results alphabetically)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS
NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON
e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: RECURSIVE CTE traverses a hierarchy.

Date & Time

Easy Q926. Find orders placed today. (order results alphabetically)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: DATE() strips time part.

Medium Q927. Find the busiest day of the week for orders. (order results alphabetically)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt
DESC LIMIT 1;
```

Tip: DAYNAME() + GROUP BY.

Hard Q928. Find customers who have not placed an order in over 6 months. (order results alphabetically)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy Q929. Find customers whose names contain exactly 4 characters. (order results alphabetically)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium Q930. Find product names that are palindromes. (order results alphabetically)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q931. Return 'Yes' if an order is above \$500, else 'No'. (order results alphabetically)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q932. Update the stock quantity of a product after a sale. (order results alphabetically)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q933. Create a view that shows each customer's total order value. (order results alphabetically)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c  
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q934. Find columns with more than 5% NULL values (generic pattern). (order results alphabetically)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable  
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium

Q935. Find all managers who have more than 5 direct reports. (order results alphabetically)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY  
manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard

Q936. Find customers who ordered the same product more than once. (order results alphabetically)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id =  
oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q937. Find the product with the most total units ordered. (order results alphabetically)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q938. Rank each order within a customer by amount, largest first. (order results alphabetically)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q939. Find products in the top 25% price within their category. (order results alphabetically)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q940. Find employees whose names contain two consecutive vowels. (order results alphabetically)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q941. Bucket orders into quartiles by amount using NTILE. (order results alphabetically)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q942. Archive old orders to an archive table and delete from main. (order results alphabetically)


```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard

Q943. Write a stored procedure to raise salary by a given percentage. (order results alphabetically)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy

Q944. List all employees including those with no department, showing NULL as 'No Dept'. (limit to 10 results)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium

Q945. Find employees who share the same salary as at least one other employee. (limit to 10 results)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard

Q946. Find customers who ordered but live in countries with fewer than 3 total customers. (limit to 10 results)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy

Q947. Find the total number of products per category. (limit to 10 results)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium

Q948. Find categories with total revenue above \$20,000. (limit to 10 results)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q949. Find the department with the greatest salary variance. (limit to 10 results)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: `VARIANCE()` aggregate function.

Window Functions

Easy

Q950. Number each order within a customer partition by date. (limit to 10 results)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: `ROW_NUMBER` with `PARTITION BY`.

Medium

Q951. Find the running average salary as employees are scanned by hire date. (limit to 10 results)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running `AVG` with cumulative window.

Hard

Q952. Calculate the salary percentile rank for each employee. (limit to 10 results)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: `PERCENT_RANK()` returns 0 to 1.

CTEs & Subqueries

Easy

Q953. Using a CTE, list all customers who live in France. (limit to 10 results)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q954. Write a recursive CTE to show an employee org chart. (limit to 10 results)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: `RECURSIVE` CTE traverses a hierarchy.

Date & Time

Easy

Q955. Find orders placed today. (limit to 10 results)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: `DATE()` strips time part.

Medium

Q956. Find the busiest day of the week for orders. (limit to 10 results)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: `DAYNAME()` + `GROUP BY`.

Hard

Q957. Find customers who have not placed an order in over 6 months. (limit to 10 results)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q958. Find customers whose names contain exactly 4 characters. (limit to 10 results)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q959. Find product names that are palindromes. (limit to 10 results)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q960. Return 'Yes' if an order is above \$500, else 'No'. (limit to 10 results)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q961. Update the stock quantity of a product after a sale. (limit to 10 results)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q962. Create a view that shows each customer's total order value. (limit to 10 results)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q963. Find columns with more than 5% NULL values (generic pattern). (limit to 10 results)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium Q964. Find all managers who have more than 5 direct reports. (limit to 10 results)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard Q965. Find customers who ordered the same product more than once. (limit to 10 results)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium Q966. Find the product with the most total units ordered. (limit to 10 results)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium Q967. Rank each order within a customer by amount, largest first. (limit to 10 results)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard Q968. Find products in the top 25% price within their category. (limit to 10 results)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium Q969. Find employees whose names contain two consecutive vowels. (limit to 10 results)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium Q970. Bucket orders into quartiles by amount using NTILE. (limit to 10 results)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard Q971. Archive old orders to an archive table and delete from main. (limit to 10 results)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.

Schema & Indexes

Hard Q972. Write a stored procedure to raise salary by a given percentage. (limit to 10 results)

```
DELIMITER // CREATE PROCEDURE raise_salary( IN dept INT, IN pct DECIMAL(5,2) ) BEGIN UPDATE Employees SET salary = salary * (1 + pct/100) WHERE dept_id = dept; END // DELIMITER ;
```

Tip: Stored procedure with IN parameter.

JOINS

Easy Q973. List all employees including those with no department, showing NULL as 'No Dept'. (use a CTE)

```
SELECT e.name, COALESCE(d.dept_name, 'No Dept') AS dept FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.dept_id;
```

Tip: LEFT JOIN + COALESCE for NULL dept.

Medium Q974. Find employees who share the same salary as at least one other employee. (use a CTE)

```
SELECT a.name, b.name, a.salary FROM Employees a JOIN Employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

Tip: Self-join on salary with < to avoid duplicates.

Hard Q975. Find customers who ordered but live in countries with fewer than 3 total customers. (use a CTE)

```
SELECT DISTINCT c.first_name, c.country FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id WHERE c.country IN ( SELECT country FROM Customers GROUP BY country HAVING COUNT(*) < 3 );
```

Tip: Subquery for small countries, then filter.

GROUP BY / HAVING

Easy Q976. Find the total number of products per category. (use a CTE)

```
SELECT category, COUNT(*) AS total FROM Products GROUP BY category;
```

Tip: Basic COUNT per group.

Medium Q977. Find categories with total revenue above \$20,000. (use a CTE)

```
SELECT p.category, SUM(o.total_amount) AS revenue FROM Products p JOIN OrderItems oi ON p.prod_id = oi.prod_id JOIN Orders o ON oi.order_id = o.order_id GROUP BY p.category HAVING SUM(o.total_amount) > 20000;
```

Tip: Multi-join + HAVING on SUM.

Hard

Q978. Find the department with the greatest salary variance. (use a CTE)

```
SELECT dept_id, VARIANCE(salary) AS sal_variance FROM Employees GROUP BY dept_id ORDER BY sal_variance DESC LIMIT 1;
```

Tip: `VARIANCE()` aggregate function.

Window Functions

Easy

Q979. Number each order within a customer partition by date. (use a CTE)

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS order_seq FROM Orders;
```

Tip: `ROW_NUMBER` with `PARTITION BY`.

Medium

Q980. Find the running average salary as employees are scanned by hire date. (use a CTE)

```
SELECT name, salary, hire_date, AVG(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS running_avg FROM Employees;
```

Tip: Running `AVG` with cumulative window.

Hard

Q981. Calculate the salary percentile rank for each employee. (use a CTE)

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank FROM Employees;
```

Tip: `PERCENT_RANK()` returns 0 to 1.

CTEs & Subqueries

Easy

Q982. Using a CTE, list all customers who live in France. (use a CTE)

```
WITH french AS ( SELECT * FROM Customers WHERE country = 'France' ) SELECT * FROM french;
```

Tip: Trivial CTE for readability.

Hard

Q983. Write a recursive CTE to show an employee org chart. (use a CTE)

```
WITH RECURSIVE org AS ( SELECT emp_id, name, manager_id, 0 AS lvl FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.name, e.manager_id, o.lvl+1 FROM Employees e JOIN org o ON e.manager_id = o.emp_id ) SELECT * FROM org ORDER BY lvl, emp_id;
```

Tip: `RECURSIVE` CTE traverses a hierarchy.

Date & Time

Easy

Q984. Find orders placed today. (use a CTE)

```
SELECT * FROM Orders WHERE DATE(order_date) = CURDATE();
```

Tip: `DATE()` strips time part.

Medium

Q985. Find the busiest day of the week for orders. (use a CTE)

```
SELECT DAYNAME(order_date) AS day_name, COUNT(*) AS cnt FROM Orders GROUP BY day_name ORDER BY cnt DESC LIMIT 1;
```

Tip: `DAYNAME()` + `GROUP BY`.

Hard

Q986. Find customers who have not placed an order in over 6 months. (use a CTE)

```
SELECT c.first_name FROM Customers c LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id
HAVING MAX(o.order_date) < CURDATE()-INTERVAL 6 MONTH OR MAX(o.order_date) IS NULL;
```

Tip: HAVING on MAX date.

String Functions

Easy

Q987. Find customers whose names contain exactly 4 characters. (use a CTE)

```
SELECT * FROM Customers WHERE CHAR_LENGTH(first_name) = 4;
```

Tip: CHAR_LENGTH equality check.

Medium

Q988. Find product names that are palindromes. (use a CTE)

```
SELECT prod_name FROM Products WHERE prod_name = REVERSE(prod_name);
```

Tip: Compare to REVERSE().

CASE & Logic

Easy

Q989. Return 'Yes' if an order is above \$500, else 'No'. (use a CTE)

```
SELECT order_id, CASE WHEN total_amount > 500 THEN 'Yes' ELSE 'No' END AS above_500 FROM Orders;
```

Tip: Two-branch CASE.

DML (INSERT/UPDATE/DELETE)

Medium

Q990. Update the stock quantity of a product after a sale. (use a CTE)

```
UPDATE Products SET stock_qty = stock_qty - 1 WHERE prod_id = 101 AND stock_qty > 0;
```

Tip: Arithmetic update with safety check.

Schema & Indexes

Medium

Q991. Create a view that shows each customer's total order value. (use a CTE)

```
CREATE VIEW customer_totals AS SELECT c.first_name, SUM(o.total_amount) AS total FROM Customers c
LEFT JOIN Orders o ON c.cust_id = o.cust_id GROUP BY c.cust_id;
```

Tip: CREATE VIEW encapsulates a query.

Data Engineering

Medium

Q992. Find columns with more than 5% NULL values (generic pattern). (use a CTE)

```
SELECT SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS null_pct FROM YourTable
HAVING null_pct > 5;
```

Tip: Conditional SUM / COUNT for NULL profiling.

JOINS

Medium

Q993. Find all managers who have more than 5 direct reports. (use a CTE)

```
SELECT manager_id, COUNT(*) AS reports FROM Employees WHERE manager_id IS NOT NULL GROUP BY manager_id HAVING COUNT(*) > 5;
```

Tip: GROUP BY manager + HAVING.

Hard

Q994. Find customers who ordered the same product more than once. (use a CTE)

```
SELECT o.cust_id, oi.prod_id, COUNT(*) AS times FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id GROUP BY o.cust_id, oi.prod_id HAVING COUNT(*) > 1;
```

Tip: Group by customer + product.

GROUP BY / HAVING

Medium

Q995. Find the product with the most total units ordered. (use a CTE)

```
SELECT prod_id, SUM(quantity) AS total_units FROM OrderItems GROUP BY prod_id ORDER BY total_units DESC LIMIT 1;
```

Tip: SUM qty, ORDER + LIMIT.

Window Functions

Medium

Q996. Rank each order within a customer by amount, largest first. (use a CTE)

```
SELECT *, RANK() OVER (PARTITION BY cust_id ORDER BY total_amount DESC) AS amount_rank FROM Orders;
```

Tip: RANK partitioned by customer.

CTEs & Subqueries

Hard

Q997. Find products in the top 25% price within their category. (use a CTE)

```
SELECT * FROM ( SELECT *, NTILE(4) OVER (PARTITION BY category ORDER BY price DESC) AS quartile FROM Products ) t WHERE quartile = 1;
```

Tip: NTILE(4) then filter quartile 1.

String Functions

Medium

Q998. Find employees whose names contain two consecutive vowels. (use a CTE)

```
SELECT * FROM Employees WHERE name REGEXP '[aeiouAEIOU]{2}';
```

Tip: REGEXP with repetition quantifier.

CASE & Logic

Medium

Q999. Bucket orders into quartiles by amount using NTILE. (use a CTE)

```
SELECT order_id, total_amount, NTILE(4) OVER (ORDER BY total_amount) AS quartile FROM Orders;
```

Tip: NTILE(4) on amount.

DML (INSERT/UPDATE/DELETE)

Hard

Q1000. Archive old orders to an archive table and delete from main. (use a CTE)

```
INSERT INTO Orders_Archive SELECT * FROM Orders WHERE order_date < '2022-01-01'; DELETE FROM Orders  
WHERE order_date < '2022-01-01';
```

Tip: INSERT...SELECT then DELETE.